

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

Tiểu luận học phần Cơ sở trí tuệ nhân tạo

Nghiên cứu một số thuật toán học máy

Sinh viên thực hiện:

Trần Hoàng Lan Phương

Giảng viên hướng dẫn: Tiến sĩ Phan Tấn Quốc

TPHCM, tháng 3 năm 2026

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

Tiểu luận học phần Cơ sở trí tuệ nhân tạo

Nghiên cứu một số thuật toán học máy

Sinh viên thực hiện:

Trần Hoàng Lan Phương

Giảng viên hướng dẫn: Tiến sĩ Phan Tấn Quốc

Thành phố Hồ Chí Minh, tháng 3 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan, bài tiểu luận là sản phẩm do tôi thực hiện tuân thủ các nguyên tắc, kết cấu của một bài tiểu luận. Là sản phẩm của cá nhân và không có bất kì sự gian lận hay sao chép nào. Các cơ sở lý luận và kiến thức được trình bày trong bài tiểu luận là hoàn toàn trung thực, có nguồn gốc rõ ràng, được chọn lọc từ nhiều nguồn tài liệu đính kèm chi tiết và hợp lệ.

Tôi xin hoàn toàn chịu trách nhiệm trước nhà trường và giảng viên hướng dẫn nếu có bất kỳ hành vi gian lận hay thông tin không trung thực nào liên quan đến nội dung của bài tiểu luận này.

MỤC LỤC

LỜI CAM ĐOAN	i
MỤC LỤC.....	ii
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT.....	v
DANH MỤC CÁC BẢNG.....	vii
DANH MỤC CÁC HÌNH VẼ	viii
LỜI MỞ ĐẦU.....	1
1. Lý do chọn đề tài	1
2. Mục đích nghiên cứu	1
3. Cấu trúc đề tài.....	2
CHƯƠNG 1. TỔNG QUAN VỀ HỌC MÁY	3
1.1. Lịch sử hình thành và phát triển	3
1.2. Định nghĩa Học máy	4
1.3. Phân loại các phương pháp học máy	6
1.3.1. Học có giám sát (Supervised Learning)	7
1.3.2. Học không giám sát (Unsupervised Learning).....	7
1.3.3. Học tăng cường (Reinforcement Learning)	8
1.4. Ứng dụng thực tiễn của Học máy trong đời sống.....	9
1.5. Tóm tắt chương 1	11
CHƯƠNG 2. CƠ SỞ TOÁN HỌC VÀ HẠ TẦNG TÍNH TOÁN	12
2.1. Mối quan hệ Dữ liệu - Thông tin - Tri thức.....	12
2.2. Cách máy tính tiếp nhận dữ liệu	13
2.2.1. Scalar (Vô hướng – Tensor bậc 0)	14
2.2.2. Vector & Embedding (Vectơ và Nhúng)	15
2.2.3. Matrix & Vai trò của GPU (Ma trận – Tensor bậc 2)	17
2.3. Tóm tắt.....	18
CHƯƠNG 3. PHÂN TÍCH CƠ CHẾ TOÁN HỌC VÀ CÁC ĐỘ ĐO ĐÁNH GIÁ MÔ HÌNH.....	19
3.1. Nhóm thuật toán Hồi quy	19

3.1.1.	Hồi quy (Regression)	19
3.1.2.	Hồi quy tuyến tính (Linear Regression)	19
3.1.3.	Hồi quy Logistic (Logistic Regression)	22
3.2.	Nhóm thuật toán Phân lớp – Random Forest.....	26
3.3.	Nhóm thuật toán Phân cụm – DBSCAN	28
3.3.1.	Thuật toán phân cụm (clustering).....	28
3.3.2.	Thuật toán DBSCAN	28
3.4.	Các độ đo đánh giá mô hình	32
3.4.1.	Đánh giá mô hình Hồi quy (Regression Metrics)	32
3.4.2.	Đánh giá một hệ thống Phân lớp (Classification Metrics).....	34
3.4.3.	Đánh giá bài toán Phân cụm (Clustering Metrics).....	37
3.5.	Tóm tắt.....	39
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ.....		40
4.1.	Mô tả bài toán và Bộ dữ liệu	40
4.1.1.	Đặt vấn đề.....	40
4.1.2.	Nguồn gốc của bộ dữ liệu.....	41
4.1.3.	Cấu trúc dữ liệu	42
4.2.	Khám phá dữ liệu (EDA).....	43
4.2.1.	Thống kê mô tả.....	43
4.2.2.	Trực quan hóa dữ liệu bộ California Housing.....	46
4.2.3.	Trực quan hóa dữ liệu bộ Credit Card Fraud Detection.....	50
4.2.4.	Làm sạch dữ liệu:	53
4.3.	Huấn luyện và Kết quả	55
4.3.1.	Thiết lập thực nghiệm.....	55
4.3.2.	Quá trình huấn luyện và Kết quả Hồi quy (California Housing)	56
4.3.3.	Quá trình huấn luyện và Kết quả Phân lớp (Fraud Detection).....	57
4.3.4.	Quá trình huấn luyện và Kết quả Phân cụm (DBSCAN).....	60
4.4.	Nhận xét.....	61
4.4.1.	So sánh hiệu năng.....	62
4.4.2.	Giải thích kết quả	62
4.4.3.	Kết luận	63
4.5.	Tóm tắt.....	63
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		64

TÀI LIỆU THAM KHẢO	65
--------------------------	----

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Tên đầy đủ (Tiếng Anh)	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
ML	Machine Learning	Học máy
ANN	Artificial Neural Network	Mạng nơ-ron nhân tạo
GPU	Graphics Processing Unit	Vi xử lý đồ họa
SVM	Support Vector Machine	Máy vector hỗ trợ
FIFO	First-In, First-Out	Nguyên tắc "vào trước ra trước"
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
DSS	Decision Support System	Hệ thống hỗ trợ đưa ra quyết định
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
MLE	Maximum Likelihood Estimation	Phương pháp Ước lượng hợp lý tối đa
MAP	Maximum A Posteriori	Ước lượng hậu nghiệm tối đa, bổ sung
i.i.d	Independent and Identically Distributed	Giả thiết các điểm dữ liệu là độc lập và cùng phân phối.
OLS	Ordinary Least Squares	Phương pháp Bình phương tối thiểu
DBSCAN	Density-Based Spatial Clustering of Applications with Noise	Thuật toán phân cụm dựa trên mật độ và có khả năng lọc nhiễu.
MSE	Mean Squared Error	Sai số bình phương trung bình
PCA	Principal Component Analysis	Phân tích thành phần chính
EDA	Exploratory Data Analysis	Quá trình khám phá và phân tích
t-SNE	t-Distributed Stochastic Neighbor Embedding	Thuật toán nhúng lân cận ngẫu nhiên phân bố T

SMOTE	Synthetic Minority Over-sampling Technique	Kỹ thuật lấy mẫu quá mức cho nhóm thiểu số tổng hợp
TN	True Negative	Dự đoán đúng (lỗi sẽ xảy ra)
TP	True Positive	Dự đoán đúng
FP	False Positive	Sai lầm loại I
FN	False Negative	Sai lầm loại II

DANH MỤC CÁC BẢNG

Bảng 4.1 Thông tin thuộc tính The California housing.....	42
Bảng 4.2 Thông tin thuộc tính Credit Card Fraud Detection	43
Bảng 4.3 So sánh hiệu năng giữa các thuật toán thực nghiệm	62

DANH MỤC CÁC HÌNH VẼ

Hình 1.1	Mối quan hệ giữa AI, Machine Learning và Deep Learning.....	4
Hình 1.2	Sơ đồ cơ chế hoạt động của các mô hình học máy.....	6
Hình 1.3	Ví dụ mô tả về phương pháp học máy có giám sát	7
Hình 1.4	Ví dụ mô tả về phương pháp học máy không giám sát	8
Hình 2.1	Liên kết dữ liệu, thông tin và tri thức	12
Hình 2.2	Mô hình hồi quy tuyến tính	14
Hình 2.3	Mô hình hệ thống Tìm kiếm ngữ nghĩa.....	16
Hình 2.4	Sự gia tăng hiệu suất (TFLOPS) khi tối ưu hóa tính toán song song ma trận.....	17
Hình 3.1	Thuật toán Hồi quy tuyến tính.....	21
Hình 3.2	Đồ thị của hàm sigmoid. Đường cong tiến đến 0 khi giá trị x giảm xuống vô cực âm và 1 khi giá trị x tăng lên vô cực.....	23
Hình 3.3	Mô hình hoạt động của thuật toán Random Forest	27
Hình 3.4	hình vẽ mô phỏng thể hiện ba loại điểm tương ứng nêu trên.....	29
Hình 3.5	Quá trình lan truyền để xác định các cụm của thuật toán DBSCAN.	31
Hình 3.6	Cách tính Precision và Recall.....	36
Hình 4.1	Mẫu 5 dòng dữ liệu mẫu đầu tiên của The California housing	44
Hình 4.2	Bảng tóm tắt thống kê The California housing	44
Hình 4.3	Mẫu 5 dòng dữ liệu từ V1 – V10 của Credit Card Fraudg.....	45
Hình 4.4	Bảng tóm tắt thống kê Credit Card Fraudng	45
Hình 4.5	Ma trận tương quan giữa các yếu tố nhà ở California.....	46
Hình 4.6	Phân phối vị trí địa lý ở California	47
Hình 4.7	Biểu đồ phân tán giữa Thu nhập và Giá nhà ở California.....	48
Hình 4.8	Biểu đồ hồi quy giữa Thu nhập và Giá nhà ở California	49
Hình 4.9	Biểu đồ Tương quan của Card Fraud Detection.....	50

Hình 4.10 Biểu đồ Boxplot so sánh 4 đặc trưng: V14, V17, V12, Amount.....	51
Hình 4.11 Biểu đồ Phân tán - t-SNE	52
Hình 4.12 Kết quả trước và sau xử lý các biến	53
Hình 4.13 Bảng dữ liệu sau khi chuẩn hóa Amount.....	54
Hình 4.14 Kết quả chia tập dữ liệu California Housing	55
Hình 4.15 Kết quả chia tập dữ liệu Credit Card Fraud Detection	56
Hình 4.16 Kết quả Hồi quy - California Housing	57
Hình 4.17 Kết quả của mô hình Random Forest	58
Hình 4.18 Ma trận nhầm lẫn của mô hình Random Forest	59
Hình 4.19 Mô hình DBSCAN	60
Hình 4.20 Kết quả phân cụm DBSCAN và thực tế (Ground Truth)	61

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Trong kỷ nguyên công nghệ số hiện nay, Học máy (Machine Learning) đã trở thành một lĩnh vực trọng yếu và đóng vai trò nền tảng trong sự phát triển của trí tuệ nhân tạo (AI). Với khả năng tự động phân tích và học hỏi từ những khối dữ liệu khổng lồ để cải thiện hiệu suất mà không cần lập trình thủ công, Học máy đóng vai trò cốt lõi trong việc tối ưu hóa quy trình vận hành của nhiều lĩnh vực từ kinh tế, y tế cho đến an ninh tài chính. Tiểu luận này tập trung nghiên cứu và thực nghiệm các thuật toán tiêu biểu thuộc các nhóm học có giám sát và không giám sát, nhằm giải quyết những bài toán thực tiễn mang tính cấp thiết trong xã hội hiện đại.

2. Mục đích nghiên cứu

Tiểu luận này được thực hiện với mục tiêu chuyển hóa những tri thức lý thuyết về trí tuệ nhân tạo thành các giải pháp ứng dụng cụ thể. Những đóng góp chính của bài nghiên cứu bao gồm:

1. Hệ thống hóa toàn diện các cơ sở lý thuyết về Học máy, từ quy trình 7 bước xây dựng hệ thống cốt lõi đến cơ chế biểu diễn dữ liệu thực tế dưới dạng các đại lượng toán học (Tensor) để tối ưu hóa hiệu suất tính toán trên phần cứng hiện đại.
2. Triển khai thực nghiệm thành công ba thuật toán chủ đạo là Hồi quy tuyến tính (Linear Regression), Rừng ngẫu nhiên (Random Forest) và DBSCAN trên các bộ dữ liệu thực tế gồm "California Housing" và "Credit Card Fraud Detection". Qua đó, tiểu luận giải quyết cụ thể bài toán dự báo giá trị bất động sản và phát hiện các hành vi gian lận tài chính tinh vi.
3. Thẩm định và đánh giá khách quan hiệu năng của các mô hình thông qua các chỉ số định lượng như MSE, R^2 , Precision-Recall và Silhouette Score. Kết quả nghiên cứu cung cấp cái nhìn chi tiết về ưu, nhược điểm của từng thuật toán, từ đó đưa ra gợi ý quan trọng cho việc lựa chọn mô hình tối ưu trong các bài toán phân tích dữ liệu thực tế.

3. Cấu trúc đề tài

Nội dung tiểu luận được trình bày trong 4 chương chính, cụ thể như sau:

Chương 1. Tổng quan về Học máy: Trình bày bức tranh toàn cảnh về lịch sử hình thành, các định nghĩa cơ bản và phân loại các phương pháp học (có giám sát, không giám sát và tăng cường). Chương này cũng nêu bật quy trình 7 bước để xây dựng một hệ thống Học máy hoàn chỉnh cùng các ứng dụng thực tiễn trong đời sống.

Chương 2. Biểu diễn dữ liệu và tri thức: Tập trung làm rõ mối quan hệ giữa dữ liệu, thông tin và tri thức trong chu trình xử lý của máy tính. Nội dung đi sâu vào cách máy tính tiếp nhận và cấu trúc hóa thế giới thực thông qua các cấp độ của Tensor (Scalar, Vector, Matrix) và vai trò của GPU trong việc tăng tốc tính toán song song.

Chương 3. Các thuật toán học máy nghiên cứu: Phân tích chi tiết logic toán học và cơ chế vận hành của các thuật toán Hồi quy (Tuyến tính, Logistic), Phân lớp (Random Forest) và Phân cụm (DBSCAN). Đồng thời, chương này thiết lập các "thước đo" chuẩn mực để đánh giá tính hiệu quả của mô hình sau khi huấn luyện.

Chương 4. Thực nghiệm và đánh giá: Đây là phần trọng tâm ứng dụng, trình bày quá trình khám phá dữ liệu (EDA), tiền xử lý chuẩn hóa thang đo và huấn luyện các mô hình trên hai bộ dữ liệu thực tế. Chương này đưa ra những nhận xét, so sánh đối chiếu hiệu năng giữa các thuật toán dựa trên kết quả thực nghiệm để rút ra kết luận cuối cùng.

CHƯƠNG 1. TỔNG QUAN VỀ HỌC MÁY

Học máy (Machine Learning - ML), đã và đang trở thành xu hướng quan trọng trong bối cảnh ứng dụng trí tuệ nhân tạo đang trở nên phổ biến trên quy mô toàn cầu. Với khả năng phân tích và học hỏi từ dữ liệu, Machine Learning cho phép máy tính tự động cải thiện hiệu suất mà không cần lập trình thủ công.

1.1. Lịch sử hình thành và phát triển

Lịch sử phát triển của Machine Learning có thể chia thành các giai đoạn gắn liền với sự bùng nổ của dữ liệu (Thủy, 1995):

Giai đoạn tiền đề (những năm 1940 - 1950): Năm 1943, mạng nơ-ron nhân tạo (ANN) đầu tiên được phác thảo bởi Walter Pitts và Warren McCulloch. Đến năm 1950, bài kiểm tra Turing (Turing Test) được đề xuất bởi Alan Turing. Năm 1959, Arthur Samuel chính thức đặt ra thuật ngữ “Machine Learning”, định nghĩa khả năng máy tính có thể học mà không cần lập trình chi tiết.

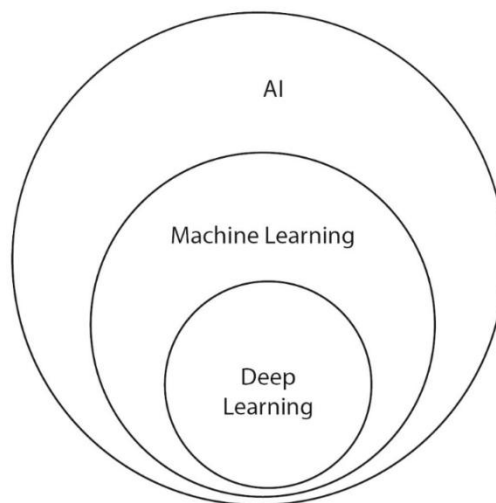
Những bước tiến sơ khai (những năm 1960 - 1980): Đánh dấu bước chuyển mình đầu tiên ta có chatbot đầu tiên (Eliza), robot thông minh di động đầu tiên (Shakey), phương tiện điều khiển từ xa bằng video (Stanford Cart), và nền tảng của học sâu (Deep Learning) đã được phát triển. Sau đó, giai đoạn những năm 1970 – 1980, những chương trình nhận dạng mẫu và ký tự viết tay, giải quyết vấn đề dựa trên chọn lọc tự nhiên, tìm kiếm hành động phụ hợp, tạo ra các quy tắc để loại bỏ các thông tin không cần thiết. Đặc biệt, vào năm 1986, thuật toán lan truyền ngược (Backpropagation) được phổ biến, giúp mạng nơ-ron có thể tự điều chỉnh lỗi. Dù vậy, giai đoạn này các thuật toán còn hạn chế do thiếu dữ liệu và sức mạnh tính toán.

Giai đoạn chuyển mình (1990 – 2000), Backprop cùng máy tính mạnh hơn đánh dấu sự phát triển trở lại của các thuật toán nơ-ron, tuy nhiên tiến trình này từng bị hạn chế do thiếu hụt dữ liệu và sức mạnh phần cứng vì sự thiếu dữ liệu và phần cứng yếu, huấn luyện chậm. Để kết thúc 10 năm “Mùa đông AI” này, cộng đồng nghiên cứu đã chuyển hướng sang học máy thống kê, đặc biệt là máy vector hỗ trợ (SVM) giúp tiến hành nhanh hơn.

Cuộc cách mạng Học sâu (2010 đến nay), cuộc cách mạng này đã tác động rất lớn đến lĩnh vực học máy, cùng với sự xuất hiện của Big Data và vi xử lý đồ họa (GPU). Đã góp phần hồi sinh mạng nơ-ron mạnh mẽ. Việc ứng dụng các kỹ thuật học sâu đã cho phép máy móc giải quyết những thách thức ngày càng phức tạp, cuối cùng dẫn đến sự ra đời của: Hệ thống nhận dạng giọng nói chính xác hơn, hệ thống đề xuất được cải thiện, xe tự lái hiệu quả hơn, và việc tạo nội dung, hình ảnh thông qua sự dân chủ hóa các công cụ AI đã xuất hiện trong thập kỷ qua.

1.2. Định nghĩa Học máy

“Machine Learning là một tập con của Trí tuệ nhân tạo (AI), tập trung vào việc xây dựng các thuật toán và mô hình cho phép máy tính học từ dữ liệu để thực hiện nhiệm vụ thay cho lập trình cứng. Bên cạnh đó, các chức năng của Học máy đang thay đổi việc sáng tạo và tương tác cũng như cách các doanh nghiệp vận hành”. (Bidgoli, 2003)



Hình 1.1 Mối quan hệ giữa AI, Machine Learning và Deep Learning

Mặc dù các phương pháp và thuật toán học máy có thể khác nhau, hầu hết các hệ thống đều được xây dựng tuân theo một quy trình cơ bản chung. Quy trình này bắt đầu từ việc thu thập dữ liệu và kết thúc bằng khả năng tự động nhận diện mẫu để đưa ra dự đoán. Để phát triển một hệ thống Học máy hoàn chỉnh, các nhà nghiên cứu cần thực hiện qua bảy bước cốt lõi sau (Bishop, 2006):

Bước 1: Thu thập dữ liệu (Data Collection)

Quy trình học máy bắt đầu bằng việc thu thập dữ liệu từ nhiều nguồn khác nhau như hệ thống giao dịch, hành vi người dùng, cảm biến, hồ sơ y tế hoặc các nền tảng trực tuyến.

Vì máy móc sẽ tự động học hỏi từ dữ liệu được cung cấp, việc có một tập dữ liệu đủ tin cậy và số lượng đủ lớn là điều kiện tiên quyết để hệ thống tìm ra các mẫu chính xác. Một bộ dữ liệu tốt là bộ dữ liệu chứa đầy đủ thông tin cần thiết, lặp lại liên tục và thể hiện rõ sự khác biệt giữa các nhóm dữ liệu. Dữ liệu càng đầy đủ, đa dạng và liên quan thì mô hình càng có cơ sở để phát hiện quy luật, và độ chính xác của dữ liệu đầu vào sẽ quyết định trực tiếp đến chất lượng của mô hình.

Bước 2: Tiền xử lý và chuẩn bị lý dữ liệu (Preprocessing & Data Preparation)

Dữ liệu thô thường chứa nhiều lỗi, thiếu thông tin hoặc không đồng nhất. Vì vậy, bước tiền xử lý nhằm làm sạch dữ liệu bằng cách loại bỏ các phần tử trùng lặp, xử lý giá trị thiếu và chuẩn hóa dữ liệu số. Bên cạnh đó, dữ liệu cần được sắp xếp lại và ngẫu nhiên hóa để phân bố đồng đều, đảm bảo thứ tự ban đầu không làm ảnh hưởng đến quá trình học. Các dữ liệu phân loại cũng được chuyển đổi sang định dạng phù hợp; thậm chí, việc cấu trúc lại tệp dữ liệu theo hàng hoặc cột là cần thiết để dữ liệu có thể được trực quan hóa và mô hình xử lý hiệu quả hơn.

Bước 3: Lựa chọn mô hình (Model Selection)

Sau khi dữ liệu đã được sàng lọc và chuẩn bị, một mô hình phù hợp sẽ được lựa chọn dựa trên mục tiêu của bài toán. Tính đến nay, khoa học dữ liệu đã phát triển đa dạng các mô hình cho từng tác vụ chuyên biệt như nhận dạng giọng nói hay nhận dạng hình ảnh. Việc xem xét kỹ lưỡng sự phù hợp của mô hình với loại dữ liệu (dữ liệu số hay phân loại) sẽ giúp cải thiện đáng kể độ chính xác và khả năng tổng quát hóa của toàn hệ thống.

Bước 4: Huấn luyện (Training)

Đây là một trong những giai đoạn quan trọng nhất để hình thành nên "trí tuệ" của máy. Trong quá trình này, dữ liệu đã chuẩn bị được truyền vào mô hình để hệ thống học các mối quan hệ và mẫu ẩn. Thông qua việc tự động điều chỉnh các tham số nội bộ, mô hình dần hoàn thành các nhiệm vụ được đặt ra. Theo thời gian đào tạo và cọ xát với dữ liệu, khả năng đưa ra quyết định của mô hình sẽ ngày càng chính xác và tối ưu hơn.

Bước 5: Kiểm tra và đánh giá (Testing and Evaluation)

Sau khi huấn luyện, mô hình cần được đánh giá nghiêm ngặt để xem xét mức độ hoạt động chính xác so với kỳ vọng ban đầu. Việc đánh giá được thực hiện bằng cách chạy thử mô hình trên một tập dữ liệu mới (Test case) mà nó chưa từng được tiếp xúc trước

đó. Thông qua các chỉ số phù hợp, bước này cung cấp một thước đo khách quan về hiệu suất, tốc độ xử lý thông tin và mức độ tin cậy của hệ thống trước khi đưa vào ứng dụng thực tiễn.

Bước 6: Điều chỉnh tham số (Hyperparameter Tuning)

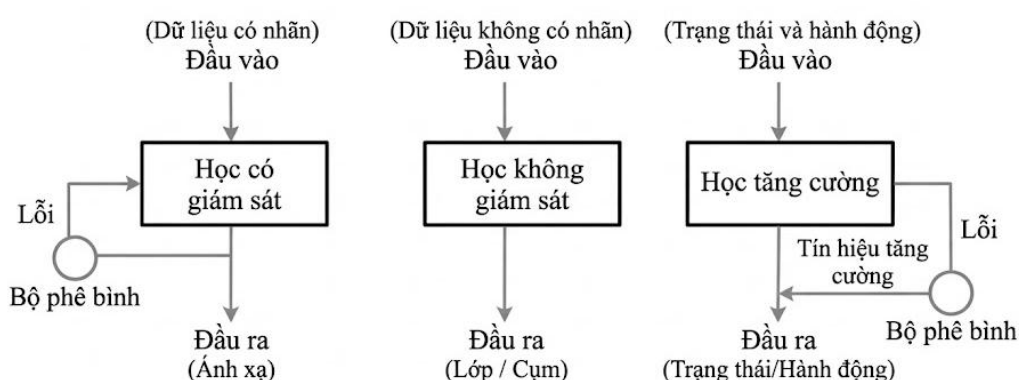
Dựa trên kết quả đánh giá ở bước 5, các tham số (các biến được lưu trong mô hình do người thiết lập quyết định) có thể được điều chỉnh lại để gia tăng hiệu suất cho hệ thống. Bằng cách thử nghiệm nhiều kịch bản, lập trình viên có thể tìm ra một con số hoặc giá trị tham số cụ thể giúp mô hình đạt được độ chính xác tối đa.

Bước 7: Đưa ra dự đoán (Prediction / Deployment)

Bước cuối cùng là triển khai mô hình Machine Learning đã hoàn thiện để phân tích thông tin thực tế và đưa ra các dự đoán một cách chính xác, phục vụ trực tiếp cho quá trình ra quyết định. Trên thực tế, để tạo ra một chương trình tối ưu nhất, các nhà nghiên cứu thường phải lặp đi lặp lại chu trình bảy bước này rất nhiều lần.

1.3. Phân loại các phương pháp học máy

Đi sâu hơn vào các vận hành của học máy. Machine Learning bắt đầu bằng việc cung cấp dữ liệu và sau đó đào tạo máy tính bằng cách xây dựng các mô hình học máy bằng cách sử dụng dữ liệu và các thuật toán khác nhau. Dựa trên mục tiêu và đặc điểm của bộ dữ liệu đầu vào, Machine Learning được chia thành ba phương pháp chính: Học giám sát (Supervised Learning), Học không giám sát (Unsupervised Learning) và Học tăng cường (Reinforcement Learning). (IS, 2025)



Hình 1.2 Sơ đồ cơ chế hoạt động của các mô hình học máy

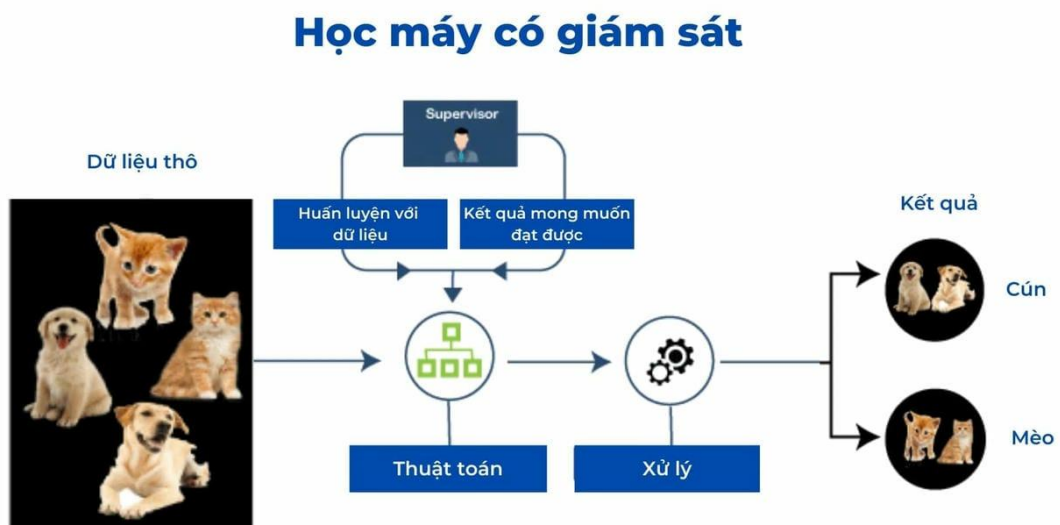
1.3.1. Học có giám sát (Supervised Learning)

Học có giám sát là mô hình học máy có bộ dữ liệu đầu vào được gọi là dữ liệu huấn luyện (training data) được gắn vào nhãn hoặc kết quả đã biết làm đầu ra. Vì vậy, nó hoạt động theo nguyên tắc cặp đầu vào-đầu ra (FIFO). Học có giám sát là loại mô hình dựa trên nhiệm vụ và tập dữ liệu phải được dán nhãn.

Ứng dụng: Phân loại email spam, nhận diện hình ảnh (ví dụ: Chó/ mèo).

Phân loại nhỏ: Phân lớp (Classification) ứng dụng trong dự đoán danh mục và Hồi quy (Regression) ứng dụng trong dự đoán giá trị số.

Ví dụ: Ta có tập dữ liệu gồm hình ảnh chó và mèo. Trong giai đoạn đầu, máy tính được huấn luyện (training data) để nhận biết hình ảnh. Khi huấn luyện hoàn tất, máy tính có thể nhận diện và dự đoán chính xác loại động vật trong hình ảnh mới được cung cấp, dựa trên việc phân tích các đặc điểm bề ngoài như hình dáng và một số đặc điểm riêng của loài. Quá trình này chính là nhận dạng đối tượng trong máy học có giám sát.



Hình 1.3 Ví dụ mô tả về phương pháp học máy có giám sát

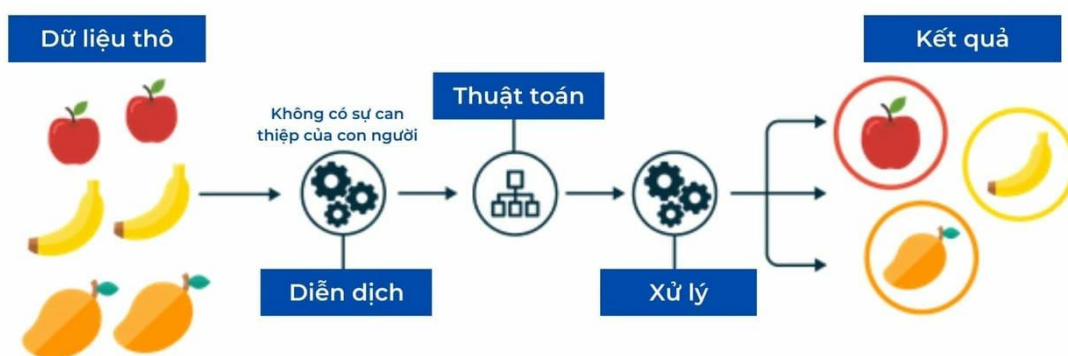
1.3.2. Học không giám sát (Unsupervised Learning)

Các mô hình Học máy không giám sát thực hiện quy trình học ngược lại với học có giám sát, có nghĩa là nó cho phép mô hình học từ tập dữ liệu huấn luyện không được gắn nhãn mà không có hướng dẫn rõ ràng. Hệ thống phải tự động tìm kiếm mô hình và cấu trúc ẩn trong chính dữ liệu. Nó được đào tạo để nhóm các đối tượng theo các đặc điểm chung.

Ứng dụng: Cá nhân hóa để xuất sản phẩm cho khách hàng dựa trên lịch sử mua sắm.

Ví dụ: Máy học không giám sát nổi bật với khả năng nhận diện các đặc điểm chung và khác biệt trong dữ liệu, làm cho nó trở thành công cụ hữu ích trong phân tích dữ liệu khám phá, phân loại khách hàng nhằm phát triển chiến lược bán chéo (Cross-sell), nhận dạng hình ảnh và nhiều ứng dụng khác.

Học máy không giám sát



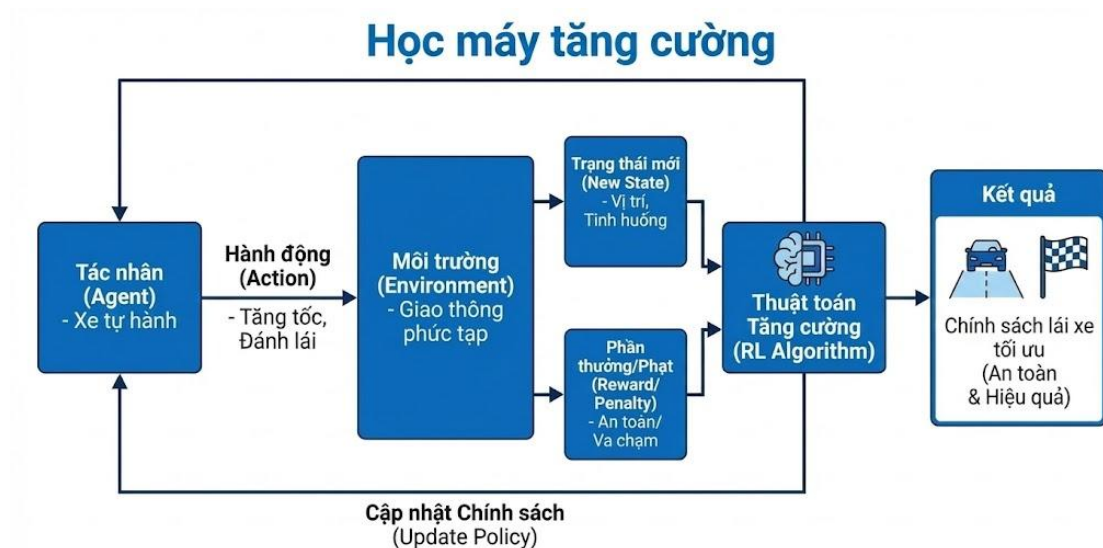
Hình 1.4 Ví dụ mô tả về phương pháp học máy không giám sát

1.3.3. Học tăng cường (Reinforcement Learning)

Trong học tăng cường, thuật toán được tạo ra để tự đào tạo bằng nhiều thử nghiệm thử và sai. Thuật toán học các hành động đối với một tập hợp các trạng thái nhất định dẫn đến trạng thái mục tiêu. Đây là mô hình học tập dựa trên phản hồi, nhận tín hiệu phản hồi dưới dạng phần thưởng (reward) hoặc hình phạt (penalty) sau mỗi cách thức mà một tác nhân học cách tương tác với môi trường (environment) đang ở một trạng thái (state) thực hiện hành động(action). Học tăng cường xảy ra khi thuật toán tương tác liên tục với môi trường thay vì dựa vào dữ liệu huấn luyện. Việc này giúp nó tìm ra được hành động tốt nhất trong từng tình huống.

Ứng dụng: Trò chơi (cờ vây), phổ biến nhất là lái xe tự động.

Ví dụ: Xe tự vận hành dựa vào học tăng cường để đưa ra các quyết định trong các tình huống giao thông phức tạp và thay đổi liên tục. Học máy tăng cường còn thông qua việc tương tác với môi trường, điều chỉnh hành động (tăng tốc, đánh lái) nhằm giảm thiểu các rủi ro (va chạm, vi phạm luật an toàn giao thông) và tối đa hóa phần thưởng (hoàn thành hành trình một cách an toàn và hiệu quả).



Hình 1.5 Ví dụ mô tả về phương pháp học máy tăng cường

1.4. Ứng dụng thực tiễn của Học máy trong đời sống

Hầu hết chúng ta tiếp xúc với Machine Learning mỗi ngày mà không hề hay biết. Từ kinh doanh, y tế cho đến công nghệ, các thuật toán Học máy hiện được tích hợp sâu vào cơ sở hạ tầng của các hệ thống phức tạp. Dưới đây là các ứng dụng tiêu biểu nhất:

Xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP)

NLP cho phép con người và máy móc có thể giao tiếp với nhau dễ dàng bằng ngôn ngữ thường ngày. Thay vì tạo ra các quy tắc lập trình từ trước, NLP dựa trên Học máy và sử dụng nhiều phương pháp thống kê máy tính tự “học” cách giao tiếp dựa trên một tập dữ liệu khổng lồ. Khi máy tính đã xử lý đủ số lượng ví dụ về ngôn ngữ người, nó sẽ xác định được các mẫu (patterns) ngữ nghĩa. Vậy nên hiệu quả của NLP tỷ lệ thuận với lượng dữ liệu đầu vào; dữ liệu càng lớn, hệ thống càng trở nên linh hoạt và thấu hiểu ngữ cảnh một cách tự nhiên.

Chatbot (Conversational AI)

Phần lớn người dùng cá nhân tương tác trực tiếp với Machine Learning dưới dạng chatbot. Thông qua việc kết hợp Học máy và Xử lý ngôn ngữ tự nhiên (NLP), Chatbot có thể học các đoạn hội thoại thực tế của con người. Thay vì chỉ đọc các tập lệnh rập khuôn, hệ thống sẽ phân tích ý định của người dùng, truy cập vào cơ sở dữ liệu có sẵn để tìm ra câu trả lời thích hợp nhất cho các truy vấn phức tạp, giúp nâng cao trải nghiệm người dùng 24/7.

Mô hình dự đoán (Predictive Modeling)

Đây là ứng dụng mang tính cốt lõi trong phân tích dữ liệu. Thuật toán ML được đào tạo dựa trên tập dữ liệu lịch sử và các biến số liên quan để phân tích thông tin hiện tại, từ đó mô phỏng nhiều tình huống có khả năng xảy ra trong tương lai với tốc độ và quy mô lớn. Sau đó, thuật toán đưa ra khuyến nghị về các hành động tối ưu nhất

Ví dụ: Trong kinh tế, ML được tích hợp vào Hệ thống hỗ trợ đưa ra quyết định (DSS-Decision Support System) giúp doanh nghiệp cắt giảm chi phí và nâng cao hiệu suất. Trong chăm sóc sức khỏe và chẩn đoán y tế, Học máy có thể ứng dụng vào việc chẩn đoán các dấu hiệu bệnh tật (như ung thư) hoặc nguy cơ biến chứng của bệnh dựa trên hồ sơ bệnh án.

Phát hiện gian lận (Fraud Detection)

Machine Learning được dùng để nhận diện các hành vi bất thường hoặc gian lận trong dữ liệu giao dịch, giúp giảm rủi ro tài chính. Các thuật toán phân loại có khả năng đọc hiểu hành vi (ví dụ: thói quen chi tiêu, vị trí địa lý của khách hàng) và ngay lập tức phát hiện ra điểm bất thường nằm ngoài mẫu chuẩn. Các hệ thống ngân hàng hiện nay sử dụng ML để phân tích đối chiếu hàng triệu dữ liệu chỉ trong vài mili-giây, từ đó quyết định chặn giao dịch hoặc gửi cảnh báo gian lận thẻ tín dụng đến khách hàng theo thời gian thực.

Hệ thống truy xuất và trích xuất thông tin (Information Extraction)

Được xây dựng dựa trên sự kết hợp giữa NLP và các công nghệ nhận dạng ký tự quang học (OCR), ứng dụng này giúp tự động xác định và trích xuất các trường thông tin quan trọng từ tài liệu thô (ví dụ: Hóa đơn, hợp đồng), ngay cả khi dữ liệu được lưu dưới dạng phi cấu trúc. Công nghệ này cũng được ứng dụng mạnh mẽ trong quy trình chuyển đổi giọng nói thành văn bản (Speech-to-Text). Việc sử dụng Học máy trong tác vụ này không chỉ nâng cao trình độ chính xác trong việc số hóa tài liệu mà còn giải phóng con người khỏi những công việc nhập liệu thủ công, lặp đi lặp lại

Ngoài những ứng dụng phổ biến nêu trên, Machine Learning còn được đưa vào giải quyết hàng loạt bài toán phức tạp khác trong kinh doanh, quản trị chuỗi cung ứng và tự động hóa, hứa hẹn mang lại nhiều giá trị hơn trong tương lai.

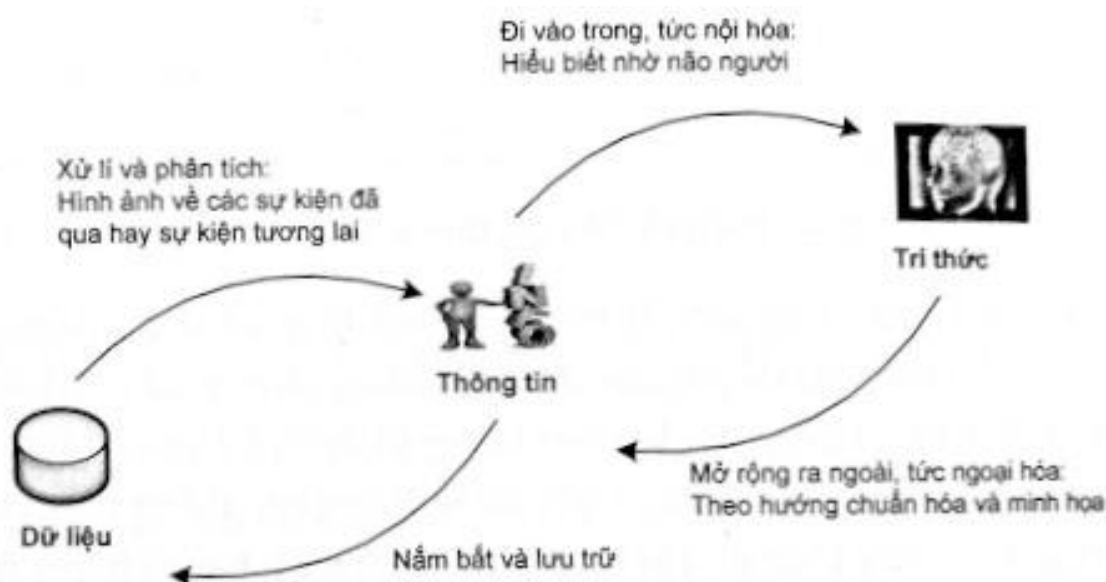
1.5. Tóm tắt chương 1

Chương 1 đã cung cấp một bức tranh toàn cảnh về Học máy (Machine Learning), từ lịch sử phát triển, định nghĩa, quy trình 7 bước xây dựng hệ thống, cho đến các phương pháp phân loại và ứng dụng thực tế. Điểm cốt lõi rút ra là: Học máy giúp máy tính tự động hóa việc ra quyết định, nhưng hiệu suất của mọi mô hình đều phụ thuộc hoàn toàn vào chất lượng dữ liệu đầu vào. Do đó, trước khi đi sâu vào logic toán học của các thuật toán, việc hiểu rõ cách máy tính tiếp nhận, cấu trúc hóa và biểu diễn dữ liệu thô là bước bản lề mang tính quyết định.

CHƯƠNG 2. CƠ SỞ TOÁN HỌC VÀ HẠ TẦNG TÍNH TOÁN

2.1. Mối quan hệ Dữ liệu - Thông tin - Tri thức

Dữ liệu (Data) là các dữ kiện thô thiếu cấu trúc; sau khi đã được tổ chức và xử lý thì ta có thông tin (Information) là những hiểu biết của chúng ta về một lĩnh vực nào đấy, dựa trên dữ liệu để biểu diễn và xử lý trong máy tính. Sau cùng, tri thức (Knowledge) là thông tin đã được phân tích, tổng hợp và áp dụng vào các ngữ cảnh cụ thể để giải quyết vấn đề hoặc đưa ra quyết định.



Hình 2.1 Liên kết dữ liệu, thông tin và tri thức

Dựa vào sơ đồ trên, ta có thể thấy mối liên hệ này không phải là một đường thẳng một chiều, mà là một chu trình chuyển hóa liên tục và khép kín:

Từ dữ liệu đến Thông tin (Xử lý và phân tích): Các dữ liệu thô (như con số, văn bản, bản ghi) ban đầu hoàn toàn vô tri. Thông qua quá trình xử lý, phân tích và gắn kết ngữ cảnh, chúng trở thành Thông tin, giúp chúng ta phác họa hình ảnh về các sự kiện đã diễn ra trong quá khứ hoặc dự báo sự kiện tương lai.

Từ Thông tin đến Tri thức (Nội hóa): Đây là quá trình đi vào trong, nơi Thông tin được tiếp thu và chuyển hóa thành sự hiểu biết sâu sắc nhờ vào não bộ con người (hoặc thông qua quá trình huấn luyện của các thuật toán AI).

Từ Tri thức về lại Thông tin (Ngoại hóa): Tri thức không chỉ dừng lại ở sự hiểu biết nội tại. Nó có thể được mở rộng ra ngoài bằng cách chuẩn hóa, minh họa thành các báo cáo, biểu đồ hoặc bộ quy tắc (rule-sets) để truyền đạt lại cho người khác hoặc hệ thống máy tính.

Từ Thông tin về lại Dữ liệu (Nắm bắt và lưu trữ): Cuối cùng, những thông tin và kết quả phân tích mới này lại tiếp tục được hệ thống nắm bắt, ghi nhận và lưu trữ ngược lại vào cơ sở dữ liệu dưới dạng Dữ liệu mới. Vòng lặp này lại tiếp tục làm giàu thêm kho tàng dữ liệu ban đầu.

Trong bối cảnh Học máy (Machine Learning), máy tính được thiết kế để tự động hóa chu trình này với tốc độ và quy mô khổng lồ. Các thuật toán đóng vai trò như “*não người*”, giúp xử lý lượng dữ liệu khổng lồ thành thông tin và từ đó tự động đúc kết thành tri thức (các mô hình dự đoán) mà không cần lập trình thủ công.

2.2. Cách máy tính tiếp nhận dữ liệu

Trước khi đi sâu vào các cấu trúc toán học, ta cần hiểu sự khác biệt căn bản trong cách con người và máy tính nhìn nhận môi trường xung quanh. Trong khi con người tiếp nhận thông tin qua các giác quan dưới dạng màu sắc, âm thanh hay văn bản, thì phần cứng máy tính lại chỉ có thể hiểu và xử lý các tín hiệu số (nhị phân). Do đó, dựa trên mối liên hệ giữa Dữ liệu – Thông tin – Tri thức, ta thấy rằng dữ liệu thô (Data) thu thập được từ thế giới thực ban đầu vốn là các dữ kiện hoàn toàn rời rạc và chưa có cấu trúc rõ ràng.

Để biến khối dữ liệu khổng lồ này thành thông tin (Information) hữu ích và xa hơn nữa là chất lọc thành dạng Tri thức (Knowledge) thông qua các thuật toán Học máy, máy tính bắt buộc phải “tiếp nhận” chúng dưới một chuẩn mực chung. Quá trình này đòi hỏi mọi đối tượng thực tế - từ một dòng thông tin khách hàng, một bức ảnh hay một đoạn ghi âm – đều được chuyển đổi thành các đại lượng toán học đa chiều có thể tính toán được, gọi chung là Tensor.

Nói một cách trực quan, Tensor đóng vai trò như một “ngôn ngữ giao tiếp” duy nhất giữa dữ liệu thực và các mô hình Học máy. Cấu trúc này tạo thuận lợi trong việc đồng nhất hóa loại định dạng dữ liệu đầu vào và còn được thiết kế tối ưu hóa cao cho các phép tính đại số tuyến tính. Nhờ việc quản lý dữ liệu dưới dạng Tensor, các framework học máy và học sâu như TensorFlow hay PyTorch mới có thể thực hiện hàng triệu phép nhân

ma trận song song. Hiểu rõ cơ chế tiếp nhận cũng chính là tiền đề để các nhà phân tích thực hiện tốt công tác tiền xử lý và trích xuất đặc trưng ở các giai đoạn sau.

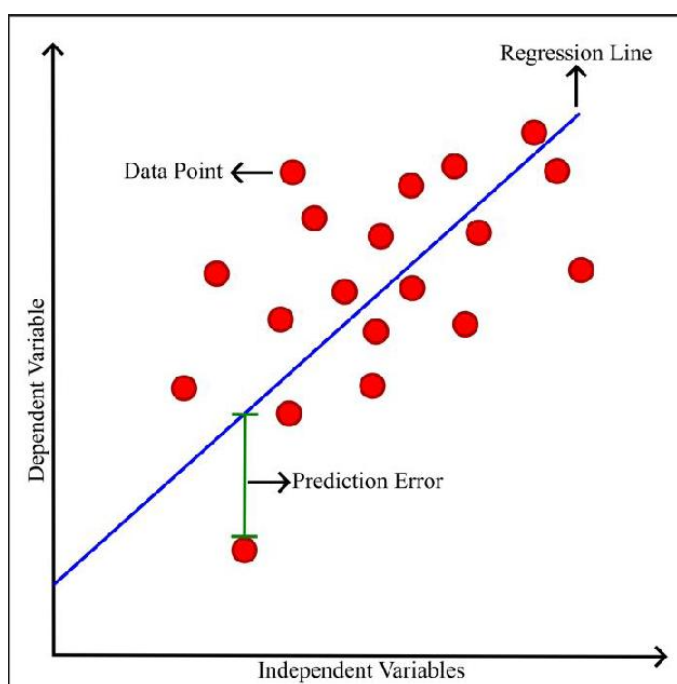
2.2.1. Scalar (Vô hướng – Tensor bậc 0)

Cách máy tính thấy: Một số thực (float) hoặc số nguyên (int) duy nhất.

Ứng dụng: Scalar thường ít được dùng làm cấu trúc dữ liệu đầu vào (input) phức tạp, mà chủ yếu đóng vai trò là kết quả đầu ra (output /target) của các bài toán hồi quy (Regression) hoặc đại diện cho giá trị của Hàm mất mát (Loss function).

Ví dụ minh họa trong phân tích Học máy:

Để hiểu rõ hơn về vai trò của Scalar, ta có thể xem xét cách nó hoạt động trong thuật toán Hồi quy tuyến tính (Linear Regression) – một thuật toán học có giám sát chuyên dùng để tạo ra và dự đoán các biến liên tục:



Hình 2.2 Mô hình hồi quy tuyến tính

Đóng vai trò là Biến dự đoán (Output Y): Trong mô hình Hồi quy tuyến tính bội (Multiple Linear Regression), giả sử một nhà phân tích dữ liệu muốn dự đoán giá cổ phiếu của tập đoàn ExxonMobil (XOM) dựa trên sự biến động của chỉ số S&P 500 và các yếu tố thị trường độc lập khác. Khi thuật toán tính toán xong phương trình:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

Kết quả đầu ra y (ví dụ: *giá cổ phiếu dự đoán là 105.5 USD*) chính là một giá trị Scalar duy nhất. Tương tự, nếu mô hình dự đoán thứ hạng của một học sinh dựa trên điểm số môn Toán, thứ hạng được trả về cũng là một đại lượng vô hướng (Scalar).

Đóng vai trò là Hàm chi phí (Cost Function): Trong quá trình tìm kiếm đường thẳng phù hợp nhất (Best Fit Line), mô hình phải liên tục đánh giá độ sai lệch giữa giá trị dự kiến và giá trị thực tế. Hàm chi phí, cụ thể là hàm Sai số Bình phương Trung bình (MSE – Mean Squared Error), sẽ cộng dồn các sai số bình phương và chia trung bình để đưa ra một thước đo tổng kết. Kết quả trả về của hàm MSE (ví dụ: $Lỗi = 0.045$) là một đại lượng Scalar. Từ một con số duy nhất này, thuật toán sẽ áp dụng phương pháp Giảm độ dốc (Gradient Descent) để liên tục điều chỉnh các trọng số cho đến khi giá trị Scalar của hàm chi phí đạt mức tối thiểu.

2.2.2. Vector & Embedding (Vector và Nhúng)

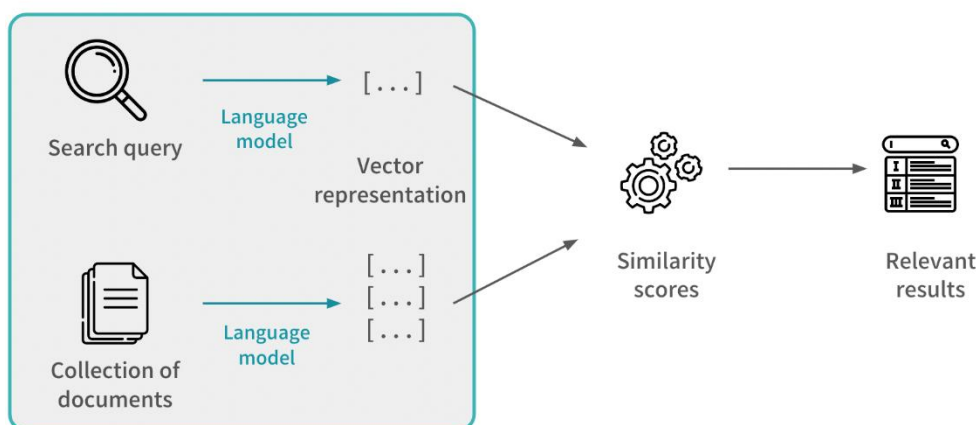
Cách máy tính thấy: Một mảng 1 chiều $[x_1, x_2, \dots, x_n]$.

Cơ chế tiếp nhận: Trước khi nạp dữ liệu vào một mô hình học máy, nó cần được “vector hóa” – chuyển đổi thành các con số đại diện cho một điểm hoặc một chuỗi điểm trong không gian vector.

Ứng dụng: Trong các bài toán hiện đại như tìm kiếm ngữ nghĩa (Semantic Search), các đối tượng như tài liệu, hình ảnh, video và âm thanh, cũng có thể được nhúng được chuyển đổi thành các Vector nhúng (Embeddings). Đây là các vector n - chiều được tạo ra bởi các thuật toán học sâu để định nghĩa ngữ nghĩa cho đầu vào. Các tác vụ trên được cải thiện đáng kể nhờ vào kỹ thuật này.

Ví dụ minh họa trong hệ thống Tìm kiếm ngữ nghĩa (Semantic Search):

Semantic search



Hình 2.3 Mô hình hệ thống Tìm kiếm ngữ nghĩa

Để hiểu rõ cách máy tính sử dụng Vector để “hiểu” dữ liệu thay vì chỉ đọc văn bản vô tri, ta có thể xem xét cách hoạt động của một hệ thống tìm kiếm ngữ nghĩa:

Biểu diễn không gian: Các mô hình nhúng sẽ biến đổi từ vựng thành các điểm trong không gian đa chiều. Những vector này không được sắp xếp ngẫu nhiên mà được tính toán sao cho các đối tượng có ý nghĩa tương đồng sẽ nằm gần nhau. Ví dụ: Vector tọa độ của từ “xe tải” và “ô tô” sẽ được đặt sát cạnh nhau.

Quy trình xử lý truy vấn: Khi người dùng tìm kiếm cụm từ “tác động của biến đổi khí hậu”, hệ thống không tìm kiếm các từ khóa khớp chính xác. Thay vào đó, nó chuyển đổi toàn bộ câu truy vấn này thành một Vector truy vấn (Query Vector). Tiếp theo, hệ thống sẽ sử dụng các hàm toán học đo lường khoảng cách giữa các vector (như khoảng cách Cosine hoặc khoảng cách Euclidean) để quét cơ sở dữ liệu và tìm ra các tài liệu có vector nằm gần với vector truy vấn gần nhất.

Kết quả tạo ra Tri thức: Nhờ quy trình tính toán khoảng cách vector này, máy tính có thể tự động trả về các tài liệu thảo luận về “ảnh hưởng của sự nóng lên toàn cầu”. Dù các từ vựng sử dụng hoàn toàn khác nhau, máy tính vẫn “hiểu” được bản chất ngữ nghĩa của chúng là tương đồng. Việc tận dụng Vector và Embedding giúp máy tính nắm bắt được ý định thực sự của người dùng, xử lý được từ đồng nghĩa và khám phá ra các mối quan hệ dữ liệu phức tạp.

2.2.3. Matrix & Vai trò của GPU (Ma trận – Tensor bậc 2)

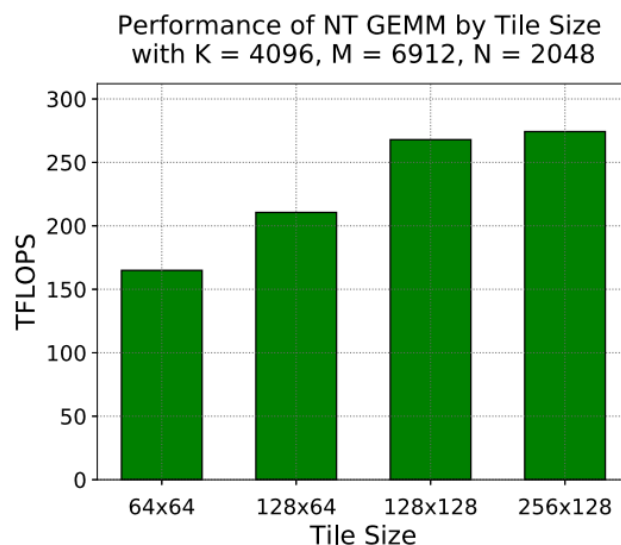
Cách máy tính nhìn thấy: Một lưới các số (sample, feature).

Cơ chế tiếp nhận: Đối với máy tính, Matrix không chỉ là bảng số liệu mà còn là cấu trúc tối ưu cho phần cứng. Các ma trận này thường được lưu trữ và xử lý trên GPU để tăng tốc độ huấn luyện và thời gian suy diễn (inference). Ngoài ra, tài liệu của PyTorch đã khẳng định: “*Sự khác biệt lớn nhất giữa một mảng NumPy và một Tensor PyTorch là Tensor PyTorch có thể chạy trên cả CPU hoặc GPU*”. Chính khả năng tương thích phần cứng này biến Tensor thành cấu trúc dữ liệu xương sống của ngành Học máy.

Ví dụ minh họa qua Phép nhân ma trận tổng quát (GEMM) trên GPU:

Để thấy rõ sự phụ thuộc của Học máy vào ma trận và GPU, ta có thể xem xét các mạng nơ-ron hoạt động. Phép nhân ma trận tổng quát (GEMM) chính là khối xây dựng nền tảng cho nhiều phép toán trong mạng nơ-ron, chẳng hạn như lớp kết nối đủ (fully-connected layers) hay mạng nơ-ron hồi quy (RNNs).

Tính toán song song: Bản chất GEMM là thực hiện phương trình $C = \alpha AB + \beta C$, đòi hỏi hàng triệu đến hàng tỷ phép toán nhân – cộng liên tiếp. Để xử lý hiệu quả khối lượng này, các GPU hiện đại không tính toán từng con số lẻ tẻ. Thay vào đó, GPU sẽ phân vùng ma trận đầu ra thành các “khối nhỏ” (tiles) và giao chúng cho các khối luồng (thread blocks) xử lý song song. (nvidia, 2023)



Hình 2.4 Sự gia tăng hiệu suất (TFLOPS) khi tối ưu hóa tính toán song song ma trận

Tối ưu hóa phần cứng bằng Lõi Tensor (Tensor Cores): Để đạt tốc độ xử lý ma trận cực hạn, các GPU thế hệ mới (như NVIDIA V100 hay A100) được trang bị Lõi Tensor chuyên dụng. Đặc biệt hiệu suất tính toán ma trận trên GPU sẽ đạt mức tối đa khi các chiều của ma trận (M, N, K) được thiết kế là bội số của 16 bytes (hoặc lên tới 128 bytes trên kiến trúc A100).

2.3. Tóm tắt

Chương 2 đã làm sáng tỏ nền tảng cốt lõi về cách máy tính tiếp nhận và xử lý dữ liệu từ môi trường xung quanh. Chương này làm rõ về việc để tự động hóa việc trích xuất tri thức, mọi đối tượng dữ liệu (hình ảnh, âm thanh và tín hiệu) để bắt buộc phải được số hóa thành dạng toán học chung là Tensor. Bằng việc đi sâu vào từng cấp độ của Tensor, ta thấy được sự gắn kết không thể tách rời giữa toán học và kỹ thuật máy tính.

CHƯƠNG 3. PHÂN TÍCH CƠ CHẾ TOÁN HỌC VÀ CÁC ĐỘ ĐO ĐÁNH GIÁ MÔ HÌNH

3.1. Nhóm thuật toán Hồi quy

3.1.1. Hồi quy (Regression)

Phân tích Hồi quy xem xét mối liên hệ giữa sự thay đổi của một hoặc nhiều biến độc lập (features) với sự thay đổi của biến phụ thuộc (target variable). Trong khoa học tự nhiên và khoa học xã hội, mục đích của hồi quy thường là mô tả mối quan hệ (characterize) giữa các đầu vào và đầu ra. Bên cạnh đó trong Học máy thường liên quan đến dự đoán. (Beers, 2026)

Thường được biết đến rộng rãi thông qua hai thuật toán nổi bật trong những thuật toán ML cơ bản cụ thể là Học giám sát (Supervised Learning): Hồi quy tuyến tính và Hồi quy Logistic.

Mối quan hệ biến đổi của ba thuật toán trên:

Như đã được biết về khái niệm chung của Hồi quy là hình dạng của đường đồ thị biểu diễn mối quan hệ giữa mỗi biến độc lập và biến phụ thuộc.

Còn trong Hồi quy tuyến tính, đường hồi quy thẳng. Bất kỳ thay đổi nào về biến độc lập đều có ảnh hưởng trực tiếp đến biến phụ thuộc. Dùng để xác định giá trị số.

Trong Hồi quy logistic, đường hồi quy là đường cong hình chữ S, còn được gọi là đường cong sigmoid. Dùng để xác định xác suất của một giá trị nhị phân hoặc phân loại

3.1.2. Hồi quy tuyến tính (Linear Regression)

Linear Regression được sử dụng rộng rãi nhất, đặc biệt trong các bài toán dự đoán giá trị liên tục. Về bản chất thuật toán cố gắng vẽ một đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều) giữa hai biến dữ liệu x và y . Là biến độc lập x được vẽ dọc theo trục hoành. Ngoài ra nó còn được gọi là biến giải thích hoặc biến dự báo. Biến phụ thuộc, y , được vẽ trên trục tung. Ta có thể tham chiếu các giá trị y như các biến phản hồi hoặc dự báo (Sharma, 2022).

Công thức toán học đơn giản $y = b_1x + b_0$ phản ánh bản chất dễ hiểu của thuật toán này, nơi b_1 là hệ số góc (coefficients) và b_0 là điểm chặn (intercept).

1. Hồi quy tuyến tính trong Học máy

Trong Học máy, các chương trình máy tính gọi là thuật toán phân tích các tập dữ liệu lớn và làm việc ngược từ dữ liệu đó để tính toán phương trình Hồi quy tuyến tính. Các nhà khoa học dữ liệu đầu tiên sẽ đào tạo thuật toán trên các tập dữ liệu đã biết hoặc được dán nhãn và sau đó sử dụng thuật toán để dự đoán các giá trị chưa biết. Dữ liệu thực tế phức tạp hơn. Đó là lý do tại sao phân tích hồi quy tuyến tính phải sửa đổi hoặc biến đổi các giá trị dữ liệu về mặt toán học để đáp ứng bốn giả định sau đây.

Mối quan hệ tuyến tính:

Để xác định mối quan hệ tuyến tính tồn tại giữa các biến độc lập và phụ thuộc, các nhà khoa học dữ liệu đã tạo ra một biểu đồ phân tán - một tập hợp ngẫu nhiên các giá trị x và y - để xem liệu chúng có nằm trên một đường thẳng hay không. Nếu không, có thể áp dụng các hàm phi tuyến tính như căn bậc hai hoặc log để tạo ra mối quan hệ tuyến tính giữa hai biến về mặt toán học.

Phần dư độc lập:

Phần dư được dùng để đo lường độ chính xác dự đoán. Phần dư là chênh lệch giữa dữ liệu quan sát được và giá trị dự đoán. Phần dư không được có một mô hình nhận dạng được giữa chúng.

Tính chuẩn:

Các kỹ thuật vẽ đồ thị như các biểu đồ $Q-Q$ xác định xem các phần dư có phân bố bình thường hay không. Các phần dư sẽ nằm dọc theo một đường chéo ở trung tâm của đồ thị. Nếu phần dư không được chuẩn hóa, ta có thể kiểm tra dữ liệu cho các điểm ngoại lai ngẫu nhiên hoặc các giá trị không phải là điển hình. Loại bỏ các điểm ngoại lai hoặc thực hiện phép biến đổi phi tuyến tính có thể khắc phục sự cố.

Phương sai không đổi:

Phương sai không đổi giả định rằng phần dư có phương sai cố định hoặc độ lệch chuẩn so với trung bình cho mọi giá trị của x . Nếu không, kết quả phân tích có thể không chính xác. Nếu giả định này chưa được đáp ứng, bạn có thể phải thay đổi biến phụ thuộc. Vì

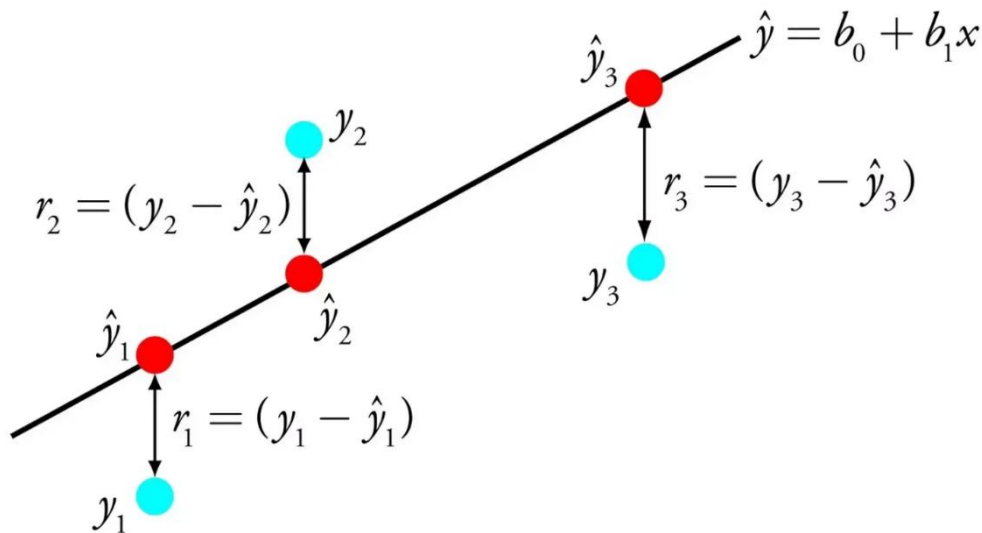
phương sai xuất hiện tự nhiên trong các tập dữ liệu lớn, giải thích được lý do phải thay đổi quy mô của biến phụ thuộc.

2. Mục tiêu tối ưu và Phương pháp Bình phương tối thiểu (OLS)

Để tìm ra đường thẳng “phù hợp nhất”, thuật toán sử dụng phương pháp Bình phương tối thiểu (Ordinary Least Squares - OLS). Mục tiêu là tìm các hệ số b sao cho *tổng bình phương sai lệch giữa giá trị dự đoán ($\hat{y}$) và giá trị thực tế (y)* là nhỏ nhất. Công thức toán học tổng quát cho mô hình đa biến (1.1. Linear Models, n.d.):

$$\hat{y}(b, x) = b_0 + b_1x_1 + \dots + b_px_p$$

Trong đó, b_0 là điểm chặn (intercept) và các giá trị $b_1, \dots, b_p$ là hệ số góc (coefficients) tương ứng với từng đặc trưng của dữ liệu.



Hình 3.1 Thuật toán Hồi quy tuyến tính

3. Vector hóa bài toán Hồi quy (Vectorization)

Trong thực tế xử lý dữ liệu lớn, việc tính toán theo từng dòng dữ liệu đơn lẻ là cực kỳ chậm. Dựa trên cấu trúc dữ liệu đã trình bày tại *Chương 2*, bài toán Hồi quy tuyến tính được “vector hóa” để máy tính có thể xử lý song song trên CPU.

Thay vì tính toán cho từng điểm dữ liệu, ta biểu diễn toàn bộ tập dữ liệu dưới dạng ma trận X (Tensor bậc 2) và các hệ số dưới dạng vector b (Tensor bậc 1). Khi đó, hàm dự đoán được viết gọn thành phép nhân ma trận:

$$\hat{y} = Xb$$

Việc vector hóa không chỉ giúp công thức toán học trở nên súc tích hơn mà còn tối ưu hóa hiệu suất thực thi trên các thư viện như NumPy hay Scikit-Learn, giúp giảm đáng kể thời gian huấn luyện mô hình đối với các bộ dữ liệu có hàng triệu bản ghi.

4. Minh họa thực hành với thư viện Scikit-Learn (Python)

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Dữ liệu: X là diện tích (m2), y là giá nhà (tỷ đồng)
X = np.array([[30], [40], [50], [60], [80]])
y = np.array([1.5, 2.1, 2.5, 3.1, 4.2])

# 2. Huấn luyện mô hình
reg = LinearRegression().fit(X, y)

# 3. Dự đoán giá cho căn nhà 70m2
house_70m2 = np.array([[70]])
predicted_price = reg.predict(house_70m2)

print(f"Trọng số (w): {reg.coef_[0]:.4f}")
print(f"Giá dự đoán cho nhà 70m2: {predicted_price[0]:.2f} tỷ đồng")

#kết quả:
#Trọng số (w): 0.0528
#Giá dự đoán cho nhà 70m2: 3.63 tỷ đồng
```

Giải thích:

- **Dữ liệu đầu vào:** Chúng ta cung cấp cho máy tính một bộ dữ liệu gồm diện tích (biến độc lập x) và giá nhà tương ứng (biến phụ thuộc y).
- **Cơ chế học:** Máy tính sử dụng phương pháp Bình phương tối thiểu (OLS) để vẽ một đường thẳng đi qua gần các điểm dữ liệu này nhất sao cho tổng bình phương sai lệch là nhỏ nhất.
- **Ý nghĩa trọng số (w):** Con số 0.0528 cho thấy cứ mỗi $1m^2$ tăng thêm, giá nhà trung bình sẽ tăng khoảng 52.8 triệu đồng.
- **Dự báo:** Dựa trên phương trình $y = wx + b$ đã học được, máy tính thực hiện phép nhân ma trận (Vectorization) để đưa ra con số 3.63 tỷ cho căn nhà $70 m^2$.

3.1.3. Hồi quy Logistic (Logistic Regression)

Logistic Regression được sử dụng để dự đoán kết quả của một biến phụ thuộc phân loại (thường là nhị phân) dựa trên các biến độc lập. Khác với Hồi quy tuyến tính dự đoán

các giá trị số liên tục, Hồi quy Logistic dự đoán xác suất để một đối tượng thuộc về một lớp cụ thể (Ví dụ: 0 hoặc 1, Đúng hoặc Sai). (Fangfang, n.d.)

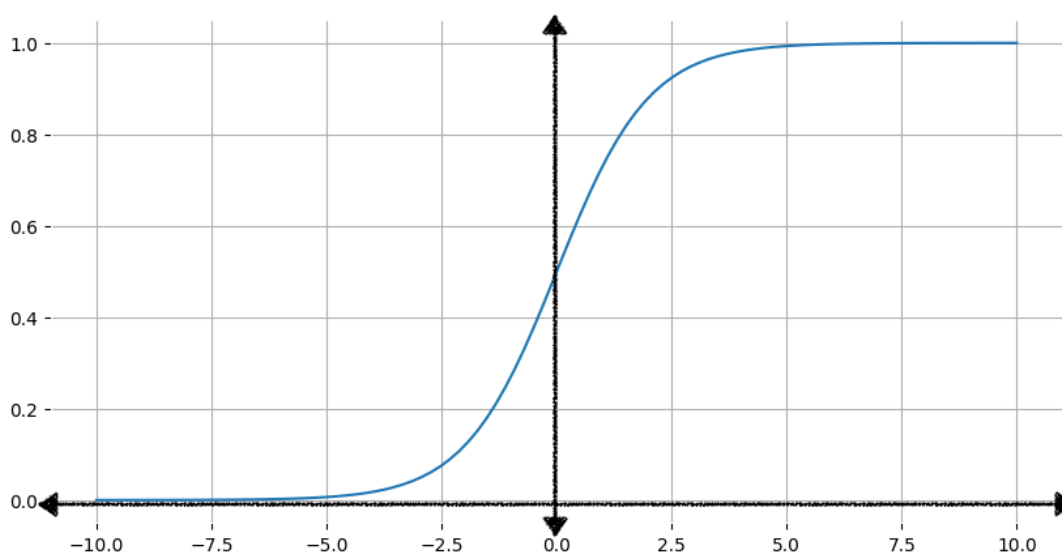
1. Bản chất hàm Sigmoid và cơ chế dự đoán xác suất

Trong thực tế, ta không thể dùng một đường thẳng để dự đoán xác suất vì giá trị xác suất phải nằm trong khoảng $[0, 1]$. Hồi quy Logistic giải quyết vấn đề này bằng cách sử dụng hàm Logistic (hàm Sigmoid) để ánh xạ mọi giá trị thực đầu vào thành một giá trị đầu ra trong khoảng từ 0 đến 1. Bên cạnh đó, đường đồ thị của hàm này có dạng đường cong hình chữ S đặc trưng.

Phương trình hàm Sigmoid (Bishop, 2006):

$$f(z) = \frac{1}{1 + e^{-z}}$$

Trong đó, z là kết quả của một tổ hợp tuyến tính các biến độc lập. Nếu $f(z) \geq 0.5$, mô hình dự đoán đối tượng thuộc lớp 1; ngược lại thuộc lớp 0.



Hình 3.1 Đồ thị của hàm sigmoid. Đường cong tiến đến 0 khi giá trị x giảm xuống vô cực âm và 1 khi giá trị x tăng lên vô cực.

2. Khái niệm Tỷ số Odds và hàm Logit

Một khái niệm then chốt trong mô hình này là Tỷ số Odds (Odds Ratio) – tỷ lệ giữa xác suất thành công (p) trên xác suất thất bại ($1-p$).

$$Odds = \frac{p}{1 - p}$$

Hàm Logit chính là logarit tự nhiên của tỷ số Odds ($\ln(Odds)$). Trong Hồi quy Logistic, Logit được mô hình hóa như một hàm tuyến tính của các biến độc lập, tạo ra "cầu nối" giữa dữ liệu thô và xác suất phân loại.

3. Tối ưu hóa bằng Ước lượng tham số: MLE và MAP

Trong việc xây dựng các mô hình thống kê và Học máy, nhiệm vụ quan trọng nhất là tìm bộ tham số tối ưu (ký hiệu là θ) đại diện cho các phân phối xác suất của dữ liệu. Để giải quyết bài toán này, hai phương pháp phổ biến nhất thường được sử dụng là Ước lượng hợp lý tối đa (MLE) và Ước lượng tối hậu tối đa (MAP).

Ước lượng hợp lý tối đa (Maximum Likelihood Estimation - MLE):

Với phương pháp MLE, việc xác định tham số θ được thực hiện bằng cách tìm các giá trị sao cho xác suất của dữ liệu huấn luyện (training data) đạt mức lớn nhất. Xác suất này còn được gọi là hàm hợp lý (likelihood):

$$\theta = \arg \max_{\theta} p(x_1, \dots, x_N | \theta)$$

Để giải bài toán tối ưu này, ta thường sử dụng giả thiết các điểm dữ liệu x_i là độc lập và cùng phân phối (i.i.d). Khi đó, bài toán MLE trở thành việc tối đa hóa tích xác suất của từng quan sát:

$$\theta = \arg \max_{\theta} \prod_{i=1}^N p(x_i | \theta)$$

Ước lượng tối hậu tối đa (Maximum A Posteriori Estimation - MAP):

Mặc dù MLE rất mạnh mẽ, nhưng nó có thể gặp hiện tượng quá khớp (overfitting) hoặc quá tự tin (overconfidence) khi tập dữ liệu huấn luyện quá nhỏ. Để khắc phục, phương pháp MAP được sử dụng bằng cách bổ sung thêm thông tin biết trước (Prior) về tham số θ , ký hiệu là $p(\theta)$.

Thay vì chỉ tối đa hóa hàm hợp lý, MAP tập trung tối đa hóa xác suất hậu nghiệm (posterior) dựa trên quy tắc Bayes:

$$\theta = \arg \max_{\theta} p(\theta | x_1, \dots, x_N)$$

$$\theta = \arg \max_{\theta} \left[\prod_{i=1}^N p(x_i | \theta) p(\theta) \right]$$

Trong đó, hàm mục tiêu chính là tích của likelihood và prior.

Ý nghĩa:

- **Thông tin ưu tiên (Prior):** Thường được chọn dựa trên các hiểu biết thực tế về tham số trước khi có dữ liệu. Một lựa chọn phổ biến là các phân phối liên hợp (conjugate distribution) giúp việc nhân thêm prior vẫn giữ được cấu trúc giống như likelihood.
- **Khả năng tổng quát hóa:** MAP có thể coi là một phương pháp giúp mô hình tránh hiện tượng quá khớp hiệu quả hơn MLE, đặc biệt là trong các trường hợp dữ liệu huấn luyện bị hạn chế, giúp đưa ra các dự đoán ổn định và chính xác hơn.

4. Vector hóa và Thực hành với Scikit-Learn

Tương tự như Hồi quy tuyến tính, bài toán này được vector hóa để tối ưu hóa tính toán trên CPU/GPU bằng cách biểu diễn đầu vào dưới dạng ma trận X và hệ số b :

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# 1. Dữ liệu: X [Số giờ học, Số bài tập], y [0: Trượt, 1: Đậu]
X = np.array([[1, 2], [2, 1], [8, 9], [9, 8]])
y = np.array([0, 0, 1, 1])

# 2. Huấn luyện mô hình
log_reg = LogisticRegression().fit(X, y)

# 3. Dự đoán xác suất đậu cho học sinh học 7 giờ và làm 8 bài tập
student_new = np.array([[7, 8]])
prob = log_reg.predict_proba(student_new)
print(f"Xác suất Đậu (Lớp 1): {prob[0][1]*100:.2f}%")
#Xác suất Đậu (Lớp 1): 99.85%
```

Giải thích:

- **Cơ chế hàm Sigmoid:** Thay vì trả về một con số bất kỳ, thuật toán nén kết quả của tổ hợp tuyến tính vào hàm Sigmoid để thu được giá trị trong khoảng $[0, 1]$.
- **Xác suất dự báo:** Với một học sinh học 7 giờ và làm 8 bài tập (gần với nhóm "Đậu" trong dữ liệu huấn luyện), mô hình tính toán và trả về mức tin cậy cực cao là 99.85%.
- **Quyết định:** Vì xác suất này > 0.5 (ngưỡng quyết định), hệ thống sẽ chính thức phân lớp học sinh này vào nhóm "Đậu" (Lớp 1).

3.2. Nhóm thuật toán Phân lớp – Random Forest

Rừng ngẫu nhiên (Random Forest) là thuật toán Học giám sát dành cho phân lớp (classification) và hồi quy (regression). (Donges, Niklas, 2024)

Khái niệm cơ bản của Random Forest là một dạng của tập hợp học (ensemble learning), nơi mà nhiều mô hình cây quyết định (phương pháp bagging), được kết hợp lại để tạo thành một mô hình mạnh mẽ hơn và đưa ra một dự đoán hoặc kết quả duy nhất, chính xác hơn.

Thuật toán có tính chất linh hoạt, dễ sử dụng và cho kết quả tuyệt vời trong hầu hết các trường hợp, ngay khi không cần tinh chỉnh siêu tham số.

Tính “Random - ngẫu nhiên” trong thuật toán:

Rừng ngẫu nhiên bổ sung thêm tính ngẫu nhiên cho mô hình trong quá trình phát triển cây. Thay vì tìm kiếm đặc điểm quan trọng nhất khi phân tách một nút, nó tìm kiếm đặc điểm tốt nhất trong một tập hợp con ngẫu nhiên các đặc điểm. Điều này dẫn đến sự đa dạng rộng lớn, thường tạo ra một mô hình tốt hơn.

Tính ngẫu nhiên của thuật toán xuất phát từ hai yếu tố chính:

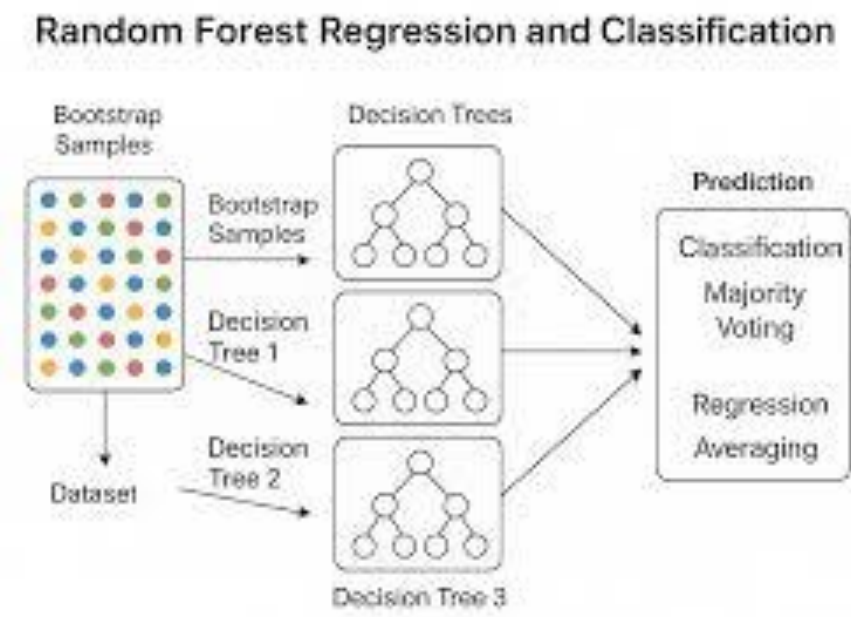
Ngẫu nhiên trong chọn mẫu:

Thay vì sử dụng toàn bộ dữ liệu huấn luyện để xây dựng từng cây quyết định, thuật toán Random Forest chọn một mẫu ngẫu nhiên từ tập dữ liệu (với hoàn lại) để xây dựng mỗi cây. Kỹ thuật này được gọi là Bagging (Bootstrap Aggregating). Bagging giúp giảm thiểu phương sai của mô hình, cải thiện độ chính xác tổng thể.

Ngẫu nhiên trong chọn đặc trưng:

Khi tạo các nút trong mỗi cây, chỉ một tập con ngẫu nhiên của tất cả các đặc trưng được xem xét để chọn đặc trưng tốt nhất tại mỗi bước. Điều này giúp các cây quyết định đa dạng hơn, giảm thiểu hiện tượng overfitting và đảm bảo rằng các cây không bị phụ thuộc quá mức vào một đặc trưng cụ thể nào đó.

Cách thức hoạt động:



Hình 3.2 Mô hình hoạt động của thuật toán Random Forest

Giả sử bộ dữ liệu đang có là n dữ liệu (*sample*) và mỗi dữ liệu có d thuộc tính (*feature*)

Để xây dựng mỗi cây quyết định cần:

1. Lấy ngẫu nhiên n dữ liệu từ bộ dữ liệu với kỹ thuật Bootstrapping, hay còn gọi là random sampling with replacement – khi ta sample được 1 dữ liệu thì ta không bỏ dữ liệu đấy ra mà vẫn giữ lại trong tập dữ liệu ban đầu, rồi tiếp tục sample cho tới khi sample đủ n dữ liệu. Khi dùng kỹ thuật này thì tập n dữ liệu mới của mình có thể có những dữ liệu bị trùng nhau.
2. Sau khi sample được n dữ liệu từ bước 1 thì mình chọn ngẫu nhiên ở k thuộc tính ($k < n$). Giờ mình được bộ dữ liệu mới gồm n dữ liệu và mỗi dữ liệu có k thuộc tính.
3. Dùng thuật toán Decision Tree để xây dựng cây quyết định với bộ dữ liệu ở bước 2.

Do quá trình xây dựng mỗi cây quyết định đều có yếu tố ngẫu nhiên (random) nên kết quả là các cây quyết định trong thuật toán Random Forest có thể khác nhau. Thuật toán Random Forest sẽ bao gồm nhiều cây quyết định, mỗi cây được xây dựng dùng thuật toán Decision Tree trên tập dữ liệu khác nhau và dùng tập thuộc tính khác nhau. Sau đó kết quả dự đoán của thuật toán Random Forest sẽ được tổng hợp từ các cây quyết định.

Khi sử dụng cần lưu ý: số lượng cây quyết định sẽ xây dựng, số lượng thuộc tính dùng để xây dựng cây. Ngoài ra, vẫn có các thuộc tính của thuật toán Decision Tree để xây dựng cây như độ sâu tối đa, số phần tử tối thiểu trong 1 node để có thể tách.

Dựa trên cách thức hoạt động ta dễ dàng nhận ra vì sao Random Forest là sự cải tiến vượt bậc so với Cây quyết định:

Cây quyết định, cho chúng ta đã thấy ưu điểm nổi bật nhất của nó là tính minh bạch và khả năng tự động tìm ra các yếu tố quan trọng. Tuy nhiên, trong môi trường dữ liệu thực tế của một nhà phân tích dữ liệu, dữ liệu thường chứa nhiều nhiễu và biến động. Một cây quyết định đơn lẻ có thể "học" cả những phần nhiễu đó, dẫn đến sai số lớn khi dự đoán dữ liệu mới (Overfitting).

Random Forest giải quyết triệt để vấn đề này. Bằng cách kết hợp nhiều cây và cho mỗi cây học trên một phần dữ liệu khác nhau, thuật toán này đảm bảo rằng các sai số của từng cây riêng lẻ sẽ bị triệt tiêu khi gộp lại. Điều này giúp mô hình trở nên "bao dung" hơn với dữ liệu nhiễu và mang lại kết quả dự báo có độ tin cậy cao hơn hẳn cho các hệ thống IT hiện đại như phát hiện gian lận hay phân tích rủi ro tín dụng.

3.3. Nhóm thuật toán Phân cụm – DBSCAN

3.3.1. Thuật toán phân cụm (clustering)

Thuật toán phân cụm (clustering): một kỹ thuật học máy không giám sát, được sử dụng để phân chia một tập dữ liệu thành các nhóm dựa trên sự tương đồng giữa các đối tượng trong cùng một nhóm. Trong thuật toán phân cụm, người dùng không cần phải cung cấp nhãn lớp cho các đối tượng trong tập dữ liệu, mà chỉ cần cung cấp một số thông tin về các đặc tính của các đối tượng.

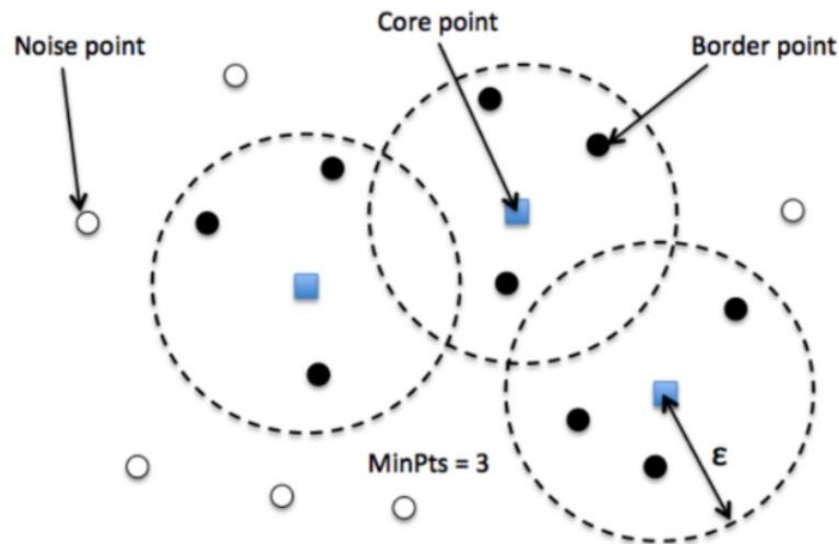
3.3.2. Thuật toán DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) - một phương pháp phân cụm dựa trên mật độ. Các thuật toán phân cụm là một công cụ quan trọng giúp tách biệt và hiểu rõ các nhóm dữ liệu khác nhau. Thuật toán hình thành các cụm bằng cách xác định các điểm trung tâm (có đủ số lượng điểm lân cận) và mở rộng chúng để tiếp cận các điểm láng giềng. Các điểm không thuộc về bất kỳ cụm nào được phân loại là nhiễu.

1. Các tham số định nghĩa mật độ

Căn cứ vào vị trí của các điểm dữ liệu so với cụm chúng ta có thể chia thành ba loại:

- Điểm nằm sâu bên trong cụm chúng ta xem chúng là *điểm lõi (core)*.
- Các *điểm biên (border)* nằm ở phần ngoài cùng của cụm
- *Điểm nhiễu (noise)* không thuộc bất kì một cụm nào.



Hình 3.3 hình vẽ mô phỏng thể hiện ba loại điểm tương ứng nêu trên

Các hình tròn đường viền nét đứt bán kính ϵ thể hiện vùng lân cận *epsilon* tương ứng với các điểm lõi nhằm xác định nhãn cho từng điểm. $minPts=3$ là số lượng tối thiểu để một điểm lõi rơi vào vùng có mật độ cao nếu xung quanh chúng có số lượng điểm tối thiểu là 3.

Trong thuật toán DBSCAN sử dụng hai tham số chính đó là:

- *minPts*: Một ngưỡng số điểm dữ liệu tối thiểu được nhóm lại với nhau nhằm xác định một vùng lân cận epsilon có mật độ cao. Số lượng *minPts* không bao gồm điểm ở tâm.
- *Epsilon (kí hiệu ϵ)*: Một giá trị khoảng cách được sử dụng để xác định vùng lân cận epsilon của bất kỳ điểm dữ liệu nào.

Hai tham số trên sẽ được sử dụng để xác định vùng lân cận *epsilon* và khả năng tiếp cận giữa các điểm dữ liệu lẫn nhau. Từ đó giúp kết nối chuỗi dữ liệu vào chung một cụm. Hai tham số trên giúp xác định ba loại điểm nêu trên.

Đối với một cặp điểm (P, Q) bất kì sẽ có ba khả năng:

- Cả P và Q đều có khả năng kết nối mật độ được với nhau. Khi đó P, Q đều thuộc về chung một cụm.
- P có khả năng kết nối mật độ được với Q nhưng Q không *kết nối mật độ* được với P . Khi đó P sẽ là điểm lõi của cụm còn Q là một điểm biên.
- P và Q đều không kết nối mật độ được với nhau. Trường hợp này P và Q sẽ rơi vào những cụm khác nhau hoặc một trong hai điểm là điểm nhiễu.

2. Các khái niệm

Để vận hành cơ chế phân cụm và phân loại dữ liệu, thuật toán DBSCAN dựa trên ba định nghĩa toán học cốt lõi sau:

Định nghĩa 1: Vùng lân cận Epsilon (Eps-neighborhood)

Vùng lân cận ϵ của một điểm dữ liệu P là tập hợp tất cả các điểm nằm trong phạm vi bán kính ϵ tính từ tâm P . Công thức xác định tập hợp này là:

$$N_{eps}(P) = \{Q \in \mathcal{D} : d(P, Q) \leq \epsilon\}$$

Trong đó $\mathcal{D}$ là tập dữ liệu và $d(P, Q)$ là hàm đo khoảng cách giữa hai điểm. Khái niệm này xác định "phạm vi ảnh hưởng" cục bộ của từng đối tượng dữ liệu.

Định nghĩa 2: Khả năng tiếp cận trực tiếp mật độ (Directly density-reachable)

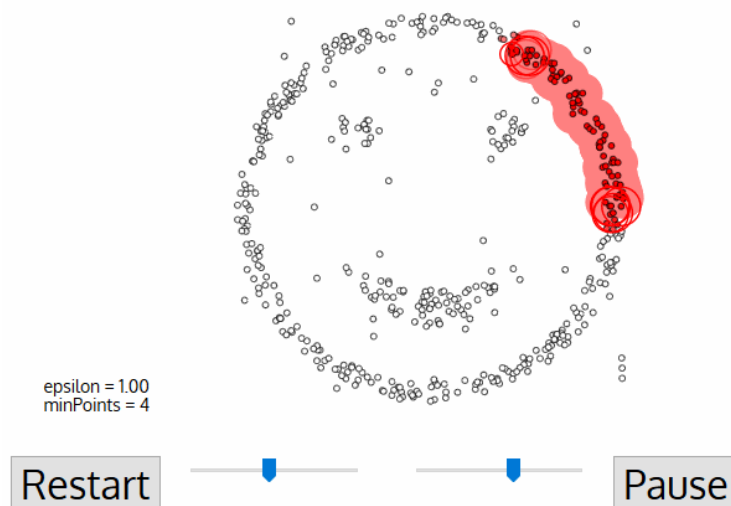
Một điểm Q được coi là có thể tiếp cận trực tiếp mật độ từ điểm P nếu nó thỏa mãn đồng thời hai điều kiện:

- Q nằm trong vùng lân cận ϵ của P ($Q \in N_{eps}(P)$)
- P là một điểm lõi (Core point), tức là số lượng điểm trong vùng lân cận của P phải đạt ngưỡng tối thiểu $|N_{eps}(P)| \geq MinPts$

Định nghĩa 3: Khả năng tiếp cận mật độ (Density-reachable) Phân loại dạng điểm

Khái niệm này mô tả khả năng hình thành một chuỗi liên kết các điểm trong cụm. Điểm Q được gọi là có thể tiếp cận mật độ từ điểm P nếu tồn tại một chuỗi các điểm: $\{P_i\}_{i=1}^n$ (với $P_1 = P$ và $P_n = Q$) sao cho mỗi điểm P_{i+1} đều có thể tiếp cận trực tiếp mật độ từ P_i .

3. Các bước



Hình 3.4 Quá trình lan truyền để xác định các cụm của thuật toán DBSCAN.

Thuật toán sẽ thực hiện lan truyền để mở rộng dần phạm vi của cụm cho tới khi chạm tới những *điểm biên* thì thuật toán sẽ chuyển sang một cụm mới và lặp lại tiếp quá trình.

Quy trình của thuật toán:

Bước 1: Thuật toán lựa chọn một điểm dữ liệu bất kì. Sau đó tiến hành xác định các *điểm lõi* và *điểm biên* thông qua vùng lân cận *epsilon* bằng cách lan truyền theo liên kết chuỗi các điểm thuộc cùng một cụm.

Bước 2: Cụm hoàn toàn được xác định khi không thể mở rộng được thêm. Khi đó lặp lại đệ qui toàn bộ quá trình với điểm khởi tạo trong số các điểm dữ liệu còn lại để xác định một cụm mới.

4. Triển khai minh họa với Scikit-Learn

```
import numpy as np
from sklearn.cluster import DBSCAN

# 1. Dữ liệu: Các tọa độ điểm (Điểm cuối cùng nằm rất xa các cụm)
X = np.array([[1, 1], [2, 2], [1.5, 1.5], [10, 10], [11, 11], [25, 80]])

# 2. Cấu hình bán kính eps=3 và số điểm tối thiểu min_samples=2
dbscan = DBSCAN(eps=3, min_samples=2)
labels = dbscan.fit_predict(X)

print(f"Nhãn cụm: {labels}")
# Kết quả: [0, 0, 0, 1, 1, -1]
```

Giải thích:

- **Kết nối mật độ:** Thuật toán bắt đầu từ các điểm lõi và lan tỏa trong bán kính $\epsilon = 3$. Ba điểm đầu tiên ở gần nhau $(1,1)$, $(2,2)$, $(1.5, 1.5)$ được kết nối mật độ và gán vào cụm 0. Hai điểm tiếp theo ở xa hơn $(10,10)$, $(11,11)$ tạo thành cụm 1.
- **Khả năng lọc nhiễu:** Điểm cuối cùng tại tọa độ $(25, 80)$ nằm hoàn toàn cô lập, không thể tiếp cận mật độ từ bất kỳ điểm nào trong bán kính ϵ .
- **Kết quả đặc biệt:** Nhãn -1 chính là bằng chứng kỹ thuật cho thấy DBSCAN đã nhận diện đây là điểm nhiễu (outlier) và loại bỏ nó khỏi các cụm chính thức, giúp dữ liệu trở nên sạch hơn.

3.4. Các độ đo đánh giá mô hình

Phần này đóng vai trò là “thước đo” khách quan để đánh giá hiệu quả của các mô hình Học máy sau khi được huấn luyện (training). Tùy thuộc vào loại bài toán, ta sẽ sử dụng các tập hợp độ đo khác nhau để đánh giá.

3.4.1. Đánh giá mô hình Hồi quy (Regression Metrics)

Để đánh giá tính hiệu quả của mô hình Hồi quy ta thông qua các mã lỗi (error metrics). Các mã lỗi này cho phép so sánh khả năng dự đoán và mức độ chính xác của mô hình.

1. Mean Squared Error (MSE)

Khái niệm:

MSE là một thước đo cho thấy sai bình phương trung bình giữa giá trị thực và giá trị dự đoán. Đây là một trong những độ đo lỗi phổ biến nhất trong các mô hình hồi quy vì nó dễ tính toán các sai số lớn do việc bình phương sai số. (Mean Squared Error, 2026)

Công thức:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Trong đó:

- y_i : Giá trị thực tế $\hat{y}_i$: Giá trị dự đoán.

Mã trận:

```
from sklearn.metrics import mean_squared_error
# Given values
Y_true = [1,1,2,2,4] # Y_true = Y (original values)
# calculated values
Y_pred = [0.6,1.29,1.99,2.69,3.4] # Y_pred = Y'
# Calculation of Mean Squared Error (MSE)
mean_squared_error(Y_true,Y_pred)
```

Giải thích:

$Y_true = [1, 1, 2, 4]$: Khai báo danh sách các giá trị thực tế (Ground Truth). Đây là những con số chính xác mà chúng ta thu thập được từ dữ liệu lịch sử.

$Y_pred = [0.6, 1.29, 1.99, 2.69, 3.4]$: Khai báo danh sách các giá trị mà mô hình đã dự đoán được sau khi huấn luyện.

$mean_squared_error(Y_true, Y_pred)$: Lệnh thực thi tính toán. Hàm này sẽ so sánh từng cặp giá trị tương ứng giữa Y_true và Y_pred để đưa ra kết quả cuối cùng.

Ý nghĩa:

MSE càng nhỏ thì mô hình càng chính xác. Tuy nhiên, vì nó bình phương sai số, các giá trị ngoại lai có thể làm MSE tăng mạnh, khiến mô hình bị phạt nhiều hơn cho những dự đoán sai lớn.

2. R-squared (R^2)

Khái niệm:

R^2 hay hệ số xác định, đo lường mức độ mô hình hồi quy có thể giải thích sự biến thiên của biến phụ thuộc. R^2 nằm trong khoảng từ 0 đến 1, với giá trị gần 1 cho thấy mô hình dự đoán tốt.

Công thức:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2}$$

Trong đó:

- y_i : Giá trị thực tế $\hat{y}_i$: Giá trị dự đoán.

Ma trận:

```
from sklearn.metrics import r2_score
# Giả sử y là giá trị thực tế và f là giá trị dự đoán từ mô hình
y = [10, 20, 30]
f = [10, 20, 30]
# Tính toán hệ số R2
r2 = r2_score(y, f)
print(f'Hệ số R2 của mô hình: {r2}')
# Đầu ra: Hệ số R2 của mô hình: 1.0
```

Ý nghĩa kết quả:

- $R^2 = 1.0$: Đây là một mô hình hoàn hảo, mọi giá trị dự đoán đều khớp hoàn toàn với giá trị thực tế. Toàn bộ sự biến động của dữ liệu đã được mô hình giải thích được.
- $R^2 = 0$: Mô hình không tốt hơn việc dự đoán bằng giá trị trung bình đơn giản của tập dữ liệu.

- R^2 âm: Dấu hiệu cho thấy mô hình của bạn đang hoạt động cực kỳ kém, thậm chí còn tệ hơn cả việc chỉ lấy giá trị trung bình để dự báo.

Ý nghĩa:

R^2 cho biết mô hình có bao nhiêu phần trăm độ biến thiên của biến mục tiêu được giải thích bởi các biến đầu vào. Giá trị càng cao (thường từ 0.7 trở lên) thì mô hình càng tốt. Tuy nhiên, R^2 không phân biệt được khi mô hình bị overfitting hoặc underfitting.

3.4.2. Đánh giá một hệ thống Phân lớp (Classification Metrics)

1. Ma trận nhầm lẫn chưa chuẩn hóa (Unnormalized Confusion matrix)

Khái niệm:

Ma trận nhầm lẫn cho ta biết được bao nhiêu phần trăm dữ liệu được phân loại đúng và chỉ ra được cụ thể mỗi loại được phân loại như thế nào, lớp nào được phân loại đúng nhiều nhất và dữ liệu thuộc lớp nào thường bị phân loại nhầm với lớp khác. Ma trận nhầm lẫn chưa chuẩn hóa là quá trình thống kê số lượng các mẫu dữ liệu dựa trên sự kết hợp giữa nhãn thực tế và nhãn dự báo.

Cấu trúc của ma trận:

Ma trận thu được là một ma trận vuông có kích thước $N \times N$, với N là số lượng lớp dữ liệu (classes).

- Hàng (Row): Đại diện cho nhãn thực tế (y_{true}).
- Cột (Column): Đại diện cho nhãn dự báo (y_{pred}).

Giá trị ô nằm ở hàng i , cột j chính là số lượng các điểm dữ liệu lẽ ra thuộc về lớp i nhưng lại được mô hình dự đoán là lớp j .

Mã nguồn:

```
def my_confusion_matrix(y_true, y_pred):
    N = np.unique(y_true).shape[0] # number of classes
    cm = np.zeros((N, N))
    for n in range(y_true.shape[0]):
        cm[y_true[n], y_pred[n]] += 1
    return cm
cnf_matrix = my_confusion_matrix(y_true, y_pred)
print('Confusion matrix:')
print(cnf_matrix)
print('\nAccuracy:', np.diagonal(cnf_matrix).sum()/cnf_matrix.sum())
```

Diễn giải:

- Xác định số lớp (N): Hàm `np.unique(y_true).shape[0]` tìm xem có bao nhiêu nhãn khác nhau trong tập dữ liệu.
- Khởi tạo ma trận không: Tạo một ma trận toàn số 0 với kích thước $N \times N$ bằng lệnh `np.zeros((N, N))`.

Vòng lặp thống kê:

- Thuật toán duyệt qua từng mẫu dữ liệu trong tập kiểm thử.
- Với mỗi mẫu thứ n , nó lấy giá trị nhãn thực làm chỉ số hàng và nhãn dự đoán làm chỉ số cột.
- `cm[y_true[n], y_pred[n]] += 1` sẽ tăng giá trị tạo ô giao nhau đó lên 1 đơn vị.

Ví dụ minh họa:

```
[[ 2.  1.  1.] <- Hàng 0 (Lớp thực tế 0)
 [ 1.  2.  0.] <- Hàng 1 (Lớp thực tế 1)
 [ 0.  1.  2.] <- Hàng 2 (Lớp thực tế 2)
```

Xét hàng đầu tiên (lớp thực tế là 0): Có tổng cộng 4 mẫu thực tế thuộc lớp 0 (2+1+1).

Trong đó:

- 2 mẫu dự đoán đúng là lớp 0 (nằm trên đường chéo chính).
- 1 mẫu bị đoán nhầm sang lớp 1.
- 1 mẫu bị đoán nhầm sang lớp 2.

Xét các phần tử trên đường chéo (2, 2, 2): Đây là tổng số mẫu được phân loại đúng của mỗi lớp. Tổng của chúng (2+2+2 = 6) dùng để tính độ chính xác (Accuracy).

Ý nghĩa:

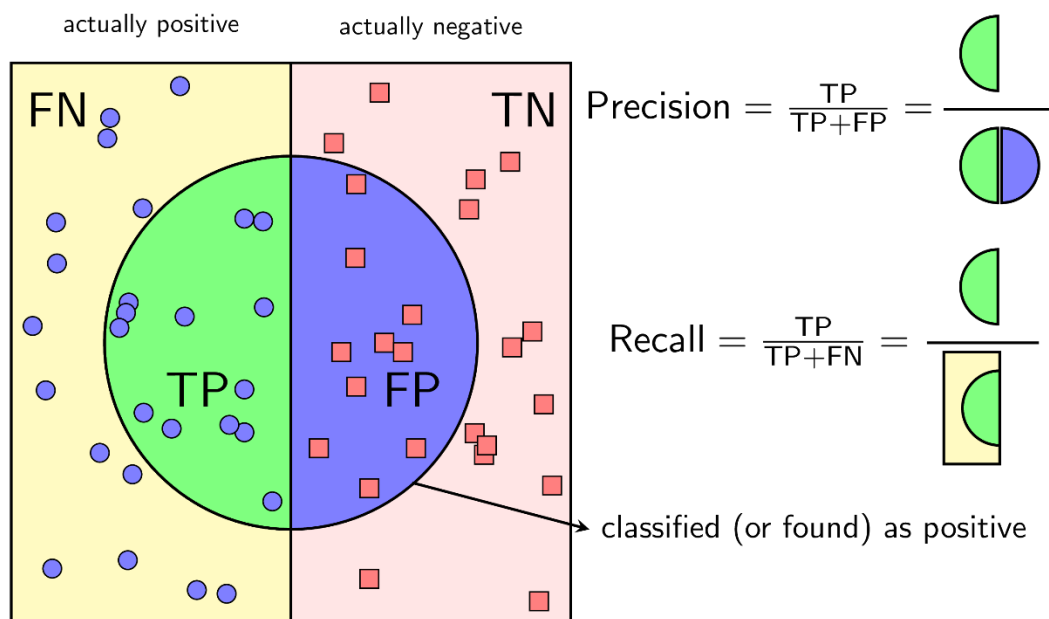
Cách làm "unnormalized" đơn thuần là đếm số lần xuất hiện của các cặp (thực tế, dự đoán). Nó cho ta biết con số cụ thể về số lượng mẫu bị nhầm lẫn, giúp nhà phân tích dữ liệu biết chính xác lớp nào đang bị mô hình phân loại sai nhiều nhất.

2. Độ chính xác chi tiết và độ nhạy (Precision and Recall)

Khái niệm:

Precision - Recall thường được ứng dụng trong các bài toán phân loại mà tập dữ liệu của các lớp chênh lệch nhau rất nhiều.

Bản chất toán học của các công thức:



Hình 3.5 Cách tính Precision và Recall.

Trong toán học, Precision và Recall là hai phân số có tử số bằng nhau nhưng mẫu số khác nhau:

Precision (Độ chính xác trên kết quả dự báo):

$$Precision = \frac{TP}{TP + FP}$$

Tử số (TP): Số lượng các điểm được dự báo đúng là *Positive*.

Mẫu số (TP+FP): Tổng số lượng điểm mà mô hình đã phân loại là *Positive* (bao gồm cả dự báo đúng và dự báo sai).

Cơ chế: Tỷ lệ số điểm thật sự đúng “*true positive*” cao đồng nghĩa các kết quả mô hình trả về có độ tin cậy lớn.

Recall (Độ nhạy trên thực tế):

$$Recall = \frac{TP}{TP + FN}$$

Tử số (TP): Số lượng các điểm được dự báo đúng là *Positive*.

Mẫu số (TP+FN): Tổng số lượng điểm thực sự là *Positive* trong tập dữ liệu (bao gồm cả điểm máy tìm thấy và điểm máy đã bỏ sót).

Cơ chế: Tỷ lệ số điểm thực sự là *Positive* cao đồng nghĩa mô hình có khả năng "quét" dữ liệu rất tốt, ít bỏ sót trường hợp quan trọng.

Mã nguồn:

Đoạn mã dưới đây minh họa cách trích xuất Precision và Recall trực tiếp từ một Ma trận nhầm lẫn cụ thể:

```
from __future__ import print_function
import numpy as np
# confusion matrix to precision + recall
def cm2pr_binary(cm):
    p = cm[0,0]/np.sum(cm[:,0])
    r = cm[0,0]/np.sum(cm[0])
    return (p, r)

# example of a confusion matrix for binary classification problem
cm = np.array([[100., 10], [20, 70]])
p,r = cm2pr_binary(cm)
print("precision = {0:.2f}, recall = {1:.2f}".format(p, r))

# Kết quả: Precision = 0.83, Recall = 0.91
```

Giải thích kết quả:

Precision = 0.83: Trong số các mẫu máy đoán là Positive, có 83% là chính xác.

Recall = 0.91: Máy đã tìm thấy được 91% tổng số lượng mẫu Positive thực tế có trong dữ liệu.

Ý nghĩa:

Một mô hình lý tưởng cần có cả hai chỉ số này càng gần 1 càng tốt. Tuy nhiên, chúng thường có xu hướng đối nghịch:

- Nếu mô hình quá "khắt khe" chỉ lấy những điểm chắc chắn nhất => Precision rất cao nhưng dễ bỏ sót nhiều điểm khác ➔ **Recall thấp**.
- Nếu mô hình quá "dễ dãi" cố gắng lấy hết mọi điểm Positive => Recall rất cao nhưng dễ lấy nhầm cả các điểm Negative ➔ **Precision thấp**.

3.4.3. Đánh giá bài toán Phân cụm (Clustering Metrics)

Điểm số Hình bóng (Silhouette Score)

Khái niệm:

Chỉ số Silhouette là một thước đo dựa trên khoảng cách hình học để đánh giá mức độ phù hợp của một quan sát với cụm được gán cho nó. Chỉ số đánh giá cả hiệu quả của các quan sát riêng lẻ lẫn chất lượng tổng thể của toàn bộ mô hình phân cụm.

Bản chất toán học của các công thức:

Để tính giá trị Silhouette tại một điểm dữ liệu p bất kỳ trong tập dữ liệu, thuật toán thực hiện qua ba bước cốt lõi:

Bước 1 - Tính độ gắn kết nội bộ (Cohesion):

Xác định $AvgDistIn(p)$: Khoảng cách trung bình giữa điểm p và tất cả các điểm khác trong cùng một cụm. Nếu cụm chỉ có duy nhất một điểm, giá trị này mặc định bằng 0.

Bước 2 - Tính độ phân tách cụm láng giềng (Separation):

Đối với từng cụm còn lại, tính khoảng cách trung bình từ p đến các điểm trong cụm đó.

Giá trị $AvgDistOut(p)$: Giá trị nhỏ nhất trong các khoảng cách trung bình này, đại diện cho khoảng cách đến "cụm láng giềng gần nhất".

Bước 3 - Tính toán chỉ số Silhouette tổng hợp:

Áp dụng công thức để chuẩn hóa kết quả về khoảng giá trị từ $[-1, 1]$:

$$s(p) = \frac{AvgDistOut(p) - AvgDistIn(p)}{\max(AvgDistOut(p), AvgDistIn(p))}$$

Mã nguồn:

```
import numpy as np
from sklearn.metrics import silhouette_score, silhouette_samples

# 1. Khai báo dữ liệu và nhãn cụm dự đoán
X = np.array([[1, 1], [2, 2], [10, 10], [11, 11]])
y_pred = np.array([0, 0, 1, 1]) # A, B thuộc cụm 0; C, D thuộc cụm 1

# 2. Tính Silhouette Score cho từng điểm riêng lẻ
sample_scores = silhouette_samples(X, y_pred)
print(f"Chỉ số Silhouette cho điểm A(1,1): {sample_scores[0]:.3f}")

# 3. Tính Silhouette Score trung bình cho toàn bộ tập dữ liệu
total_score = silhouette_score(X, y_pred)
print(f"Chỉ số Silhouette tổng thể: {total_score:.3f}")
```

Trong đó:

$$AvgDistIn(A) = \text{dist}(A, B) = 1.414$$

$$AvgDistOut(p) = \frac{\text{ddist}(A,C) + \text{dist}(A,D)}{2} = 13.435$$

$$s(p) = \frac{AvgDistOut(p) - AvgDistIn(p)}{\max(AvgDistOut(p), AvgDistIn(p))} = 0.895$$

Tương ứng với code:

Chỉ số Silhouette cho điểm A (1, 1): 0.895

Chỉ số Silhouette tổng thể: 0.890.

Ý nghĩa:

Bên cạnh việc đánh giá chất lượng từng điểm dữ liệu, chỉ số Silhouette còn được sử dụng như một phương pháp quan trọng để xác định giá trị tối ưu của số cụm k . Thay vì chọn k theo cảm tính, nhà phân tích sẽ chạy thuật toán với nhiều giá trị k khác nhau và chọn giá trị k cho điểm Silhouette trung bình cao nhất.

3.5. Tóm tắt

Việc đánh giá bằng các chỉ số định lượng là giải pháp minh bạch để kiểm tra hiệu suất mô hình trước khi đưa vào ứng dụng thực tế. Hiểu rõ cơ chế hoạt động và ý nghĩa của các độ đo giúp tối ưu hóa các thuật toán Hồi quy, Phân lớp và Phân cụm.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Mô tả bài toán và Bộ dữ liệu

4.1.1. Đặt vấn đề

Sau khi đã đi sâu vào nghiên cứu chi tiết về cơ chế vận hành cũng như nền tảng toán học nền tảng của các thuật toán Hồi quy, Phân lớp và Phân cụm tại Chương 3, một thách thức lớn được đặt ra là làm thế nào để chuyển hóa những "tri thức" lý thuyết trừu tượng này thành công cụ giải quyết hiệu quả các bài toán thực tế. Trong kỷ nguyên dữ liệu lớn (Big Data) hiện nay, hiệu suất thực sự của một mô hình học máy không chỉ dừng lại ở sự tinh vi của các công thức toán học, mà còn phụ thuộc mật thiết vào khả năng xử lý, làm sạch và thích ứng linh hoạt với các tập dữ liệu có độ nhiễu cao, quy mô khổng lồ và cấu trúc phức tạp.

Việc triển khai chương thực nghiệm này không chỉ đơn thuần là bước minh họa, mà còn nhằm giải quyết ba mục tiêu chiến lược cốt lõi:

Kiểm chứng tính đúng đắn và khả năng ứng dụng:

Thông qua việc áp dụng các thuật toán kinh điển như Hồi quy tuyến tính (Linear Regression), chúng ta hướng tới việc biến đổi các luồng dữ liệu thô (raw data) vốn rời rạc thành các mô hình dự báo có giá trị kinh tế cao và hỗ trợ các hệ thống nghiệp vụ thông minh trong việc đưa ra quyết định.

Đối chiếu giữa lý thuyết và thực hành:

Đối chiếu và thu hẹp khoảng cách giữa lý thuyết và thực hành:

Bằng cách sử dụng các bộ dữ liệu tiêu chuẩn quốc tế như “California Housing” và “Credit Card Fraud Detection”, chương thực nghiệm sẽ quan sát và phân tích cách các thư viện hiện đại (điển hình là Scikit-Learn) thực hiện quá trình tối ưu hóa tham số. Điều này giúp làm sáng tỏ mối liên hệ giữa các phương pháp ước lượng xác suất như MLE (Maximum Likelihood Estimation) hay MAP (Maximum A Posteriori) đã nghiên cứu với kết quả thực thi trên máy tính.

Đánh giá định lượng khách quan qua các hệ thống độ đo:

Việc sử dụng các chỉ số đo lường như MSE (Mean Squared Error) cho hồi quy, Precision-Recall cho các bài toán mất cân bằng dữ liệu, hay Silhouette Score cho phân cụm không chỉ giúp xác định độ chính xác của mô hình. Đây còn là giải pháp minh bạch và khoa học để thẩm định mức độ tin cậy của mô hình trước khi chuyển giao vào các hệ thống ứng dụng thực tiễn trong đời sống và sản xuất.

4.1.2. Nguồn gốc của bộ dữ liệu

Để phân tích và đánh giá hiệu năng của ba mô hình (Hồi quy tuyến tính, Rừng ngẫu nhiên và DBSCAN), nghiên cứu này sử dụng các tập dữ liệu đa dạng nhằm đảm bảo tính tương thích với đặc thù của từng thuật toán.

The California housing và Credit Card Fraud Detection là hai bộ dữ liệu phù hợp cho các thuật toán trên.

Trong đó “The California housing” là tập dữ liệu được lấy từ cuộc điều tra dân số Hoa Kỳ năm 1990, sử dụng một hàng cho mỗi cuộc điều tra dân số. Cục điều tra dân số công bố dữ liệu mẫu (một nhóm khối thường có dân số từ 600 đến 3.000 người). Hộ gia đình là một nhóm người sống trong một ngôi nhà. Kể từ khi trung bình số phòng và phòng ngủ trong bộ dữ liệu này được cung cấp cho mỗi hộ gia đình, những các cột có thể có giá trị lớn đáng ngạc nhiên đối với các nhóm khối có ít hộ gia đình và nhiều ngôi nhà trống, chẳng hạn như khu nghỉ dưỡng. (Nugent, 2018)

Còn “Credit Card Fraud Detection” là tập dữ liệu đến từ Kaggle. Tập dữ liệu được sử dụng bao gồm các giao dịch thẻ tín dụng được thực hiện bởi chủ thẻ châu Âu vào tháng 9 năm 2013. Tập dữ liệu này chứa 492 vụ gian lận trong số 284.807 giao dịch trong hai ngày. Tập dữ liệu không cân bằng, với loại tích cực (lừa đảo) chiếm 0,172% tổng số giao dịch. (Adrien Payong, Abdeladim Fadheli, 2022)

Để vận hành mô hình Hồi quy tuyến tính ta lựa chọn bộ dữ liệu “The California housing” để thực hiện dự đoán giá trị trung bình của các ngôi nhà trong một khu vực tại California. Bộ dữ liệu “Credit Card Fraud Detection” được áp dụng cho thuật toán còn lại, vì trong bài toán gian lận, ta vừa có thể phân lớp (là gian lận hay không bằng Rừng ngẫu nhiên), vừa có thể phân cụm (để tìm ra các mẫu hành vi lạ/ nhiều bằng DBSCAN).

4.1.3. Cấu trúc dữ liệu

Cấu trúc dữ liệu “The California housing”:

Số trường hợp: 20640.

Trong đó có 8 biến số (numeric) đóng vai trò là các biến dự báo (predictive attributes) và 1 biến mục tiêu (target).

Tình trạng dữ liệu: Không có giá trị thiếu (None Missing Attribute Values), giúp đảm bảo tính toàn vẹn cho quá trình huấn luyện mô hình.

Thông tin thuộc tính:

Bảng 4.1 Thông tin thuộc tính The California housing

Thứ tự	Tên thuộc tính	Ý nghĩa
1	MedInc	Thu nhập trung bình của hộ gia đình trong khu vực.
2	HouseAge	Độ tuổi trung bình của các ngôi nhà trong khu vực.
3	AveRooms	Số lượng phòng trung bình trên mỗi hộ gia đình.
4	AveBedrms	Số lượng phòng ngủ trung bình trên mỗi hộ gia đình.
5	Population	Tổng dân số sinh sống trong khu vực.
6	AveOccup	Số lượng thành viên trung bình trong một hộ gia đình.
7	Latitude	Vĩ độ địa lý của khu vực.
8	Longitude	Kinh độ địa lý của khu vực.
Đích	Target Value	Giá trị trung bình của ngôi nhà (MedHouseVal).

Cấu trúc dữ liệu “Credit Card Fraud Detection”:

Bộ dữ liệu này ghi lại các giao dịch thực hiện bằng thẻ tín dụng trong hai ngày, bao gồm:

Số trường hợp: 284.807 giao dịch.

Có 492 giao dịch gian lận (chiếm 0,172%).

Thông tin thuộc tính:

Bảng 4.2 Thông tin thuộc tính Credit Card Fraud Detection

Tên biến	Loại biến	Ý nghĩa
Time	Numerical	Số giây trôi qua giữa giao dịch hiện tại và giao dịch đầu tiên trong tập dữ liệu.
V1 – V28	Numerical	Các thành phần chính thu được từ phép biến đổi PCA nhằm bảo mật danh tính và các đặc trưng nhạy cảm của khách.
Amout	Numerical	Giá trị bằng tiền của giao dịch. Biến này có thể được dùng cho học máy nhạy cảm với chi phí (cost-sensitive learning).
Class	Binary	Biến mục tiêu (phản hồi): Giá trị 1 trong trường hợp gian lận và 0 nếu giao dịch bình thường.

4.2. Khám phá dữ liệu (EDA)

4.2.1. Thống kê mô tả

Chú thích:

Mean (Giá trị trung bình): Tổng các giá trị chia cho số lượng phần tử, đại diện cho xu hướng trung tâm của bộ dữ liệu

Std (Standard Deviation - Độ lệch chuẩn): Đo lường mức độ phân tán của dữ liệu xung quanh Mean.

Min (Minimum - Giá trị nhỏ nhất): Giá trị thấp nhất trong tập dữ liệu.

Max (Maximum - Giá trị lớn nhất): Giá trị cao nhất trong tập dữ liệu.

Mô tả dữ liệu “The California housing”:

--- 5 DÒNG ĐẦU TIÊN CỦA DỮ LIỆU ---									
	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

Hình 4.1 Mẫu 5 dòng dữ liệu mẫu đầu tiên của The California housing

--- BẢNG TÓM TẮT THỐNG KÊ ---				
	mean	std	min	max
MedInc	3.870671	1.899822	0.499900	15.000100
HouseAge	28.639486	12.585558	1.000000	52.000000
AveRooms	5.429000	2.474173	0.846154	141.909091
AveBedrms	1.096675	0.473911	0.333333	34.066667
Population	1425.476744	1132.462122	3.000000	35682.000000
AveOccup	3.070655	10.386050	0.692308	1243.333333
Latitude	35.631861	2.135952	32.540000	41.950000
Longitude	-119.569704	2.003532	-124.350000	-114.310000
MedHouseVal	2.068558	1.153956	0.149990	5.000010

Hình 4.2 Bảng tóm tắt thống kê The California housing

Tổng quan về dữ liệu:

Dựa vào bảng thống kê, ta thấy có sự chênh lệch rất lớn về khoảng cách giá trị giữa các biến. Ví dụ như biến MedInc (Thu nhập) chỉ dao động từ 0.5 – 15, trong khi Population (Dân số) có giá trị cực đại lên đến hơn 35,000.

Từ đây ta thấy được vấn đề của dữ liệu nếu áp dụng vào mô hình Hồi quy tuyến tính. Các biến có giá trị lớn (như Population) sẽ “áp đảo” và làm sai lệch kết quả của các biến có giá trị nhỏ nhưng quan trọng khác.

Để giải quyết vấn đề trên, ta cần thực hiện bước Feature Scaling (Chia tỷ lệ đối tượng) giúp chuyển đổi các đối tượng số thành một phạm vi hoặc phân bố tương, tự trước khi huấn luyện mô hình.

Mô tả dữ liệu “Credit Card Fraud Detection”:

--- 5 DÒNG ĐẦU TIÊN (ĐÃ LỌC CỘT CHO BÀI TIỂU LUẬN) ---												
	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	Amount	Class
0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794	149.62	0
1	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974	2.69	0
2	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643	378.66	0
3	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	-0.054952	123.50	0
4	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	0.753074	69.99	0

Hình 4.3 Mẫu 5 dòng dữ liệu từ V1 – V10 của Credit Card Fraudg

--- BẢNG TÓM TẮT THỐNG KÊ RÚT GỌN ---				
	mean	std	min	max
V1	0.0	1.958696	-56.40751	2.45493
V2	0.0	1.651309	-72.715728	22.057729
V3	-0.0	1.516255	-48.325589	9.382558
V4	0.0	1.415869	-5.683171	16.875344
V5	0.0	1.380247	-113.743307	34.801666
V6	0.0	1.332271	-26.160506	73.301626
V7	-0.0	1.237094	-43.557242	120.589494
V8	0.0	1.194353	-73.216718	20.007208
V9	-0.0	1.098632	-13.434066	15.594995
V10	0.0	1.08885	-24.588262	23.745136
...	...	...	...	...
V27	-0.0	0.403632	-22.565679	31.612198
V28	-0.0	0.330083	-15.430084	33.847808
Amount	88.349619	250.120109	0.0	25691.16
Class	0.001727	0.041527	0.0	1.0
--- TỶ LỆ PHÂN PHỐI LỚP (CLASS DISTRIBUTION) ---				
Giao dịch bình thường (Class 0): 99.827251%				
Giao dịch gian lận (Class 1): 0.172749%				

Hình 4.4 Bảng tóm tắt thống kê Credit Card Fraudg

Tổng quan về dữ liệu:

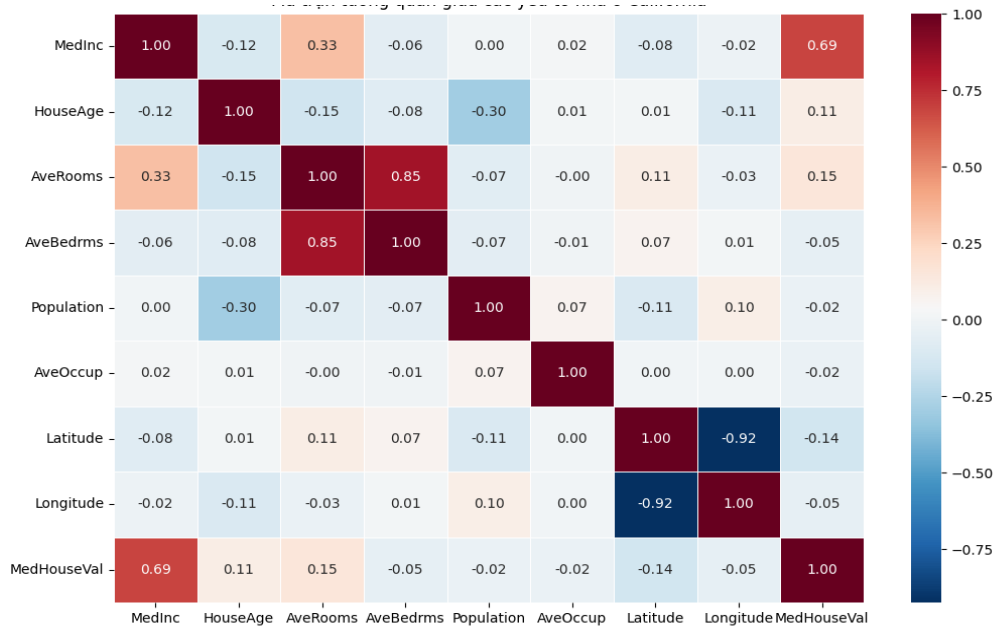
Dựa vào bảng thống kê, ta thấy có sự chênh lệch rất lớn. Trong khi các biến từ V1 đến V28 đã được chuẩn hóa (dao động quanh giá trị 0), thì biến *Amount* (Số tiền) có giá trị cực đại lên tới 25,691 còn biến *Time* lên tới 172,792. Điều này bắt buộc chúng ta phải thực hiện Feature Scaling chuyển đổi hai đối tượng số này thành một phạm vi.

Xét về tính bảo mật (PCA): Hầu hết các biến độc lập (từ V1 đến V28) là kết quả của phép biến đổi PCA. Điều này giải thích tại sao giá trị trung bình (*mean*) của chúng đều xấp xỉ bằng 0. Đây là một đặc điểm quan trọng giúp bảo mật thông tin nhạy cảm của khách hàng ngân hàng.

Xét về độ mất cân bằng: Chỉ có khoảng 0.17% giao dịch là gian lận (*Class = 1*). Đây là một tập dữ liệu cực kỳ mất cân bằng (highly imbalanced). Do đó, việc xây dựng mô hình cần đi kèm với các kỹ thuật xử lý dữ liệu đặc biệt như SMOTE hoặc điều chỉnh trọng số lớp (Class Weights) thay vì chỉ dùng các thuật toán thông thường.

4.2.2. Trực quan hóa dữ liệu bộ California Housing

1. Ma trận tương quan (Correlation Matrix)



Hình 4.5 Ma trận tương quan giữa các yếu tố nhà ở California

Biểu đồ này cực kỳ quan trọng trong giai đoạn Feature Selection (chọn lọc đặc trưng) của các thuật toán ML.

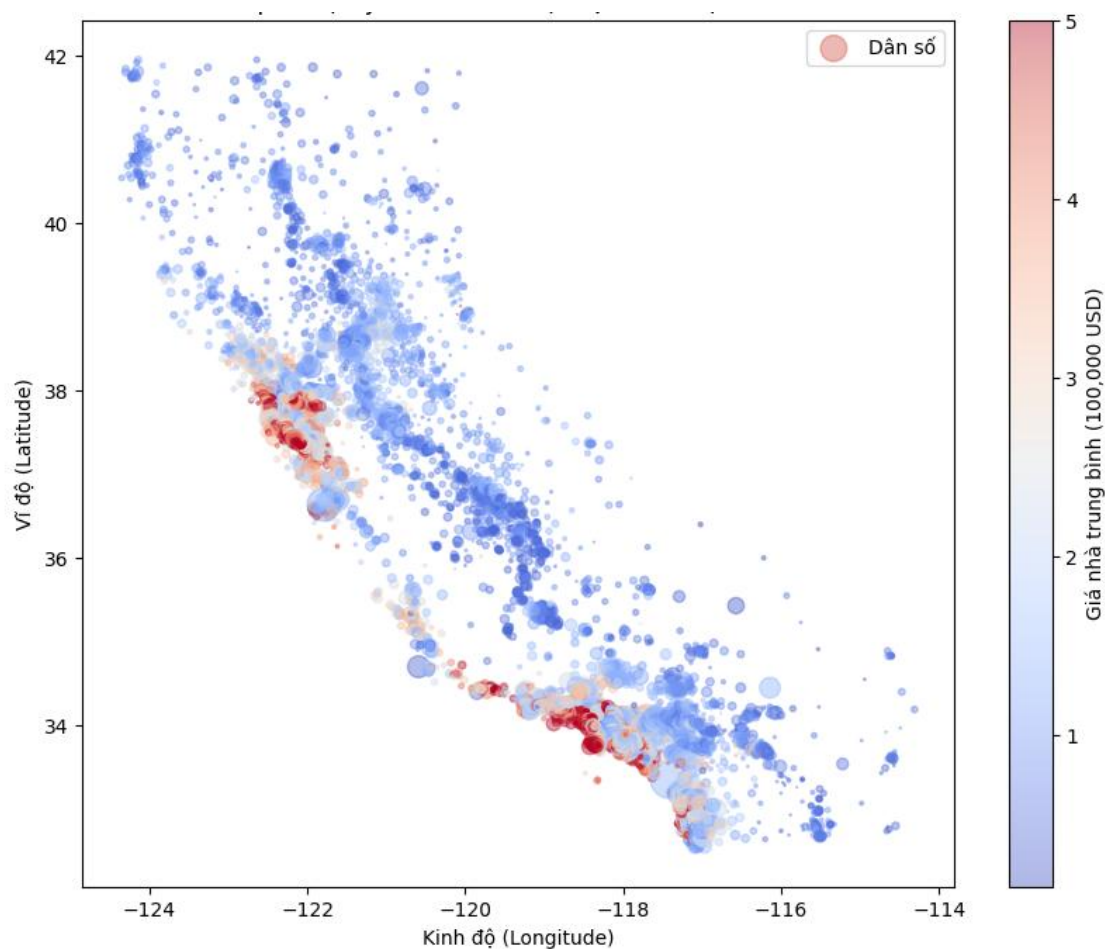
Mối tương quan mạnh nhất: Thu nhập trung bình (MedInc) có mối tương quan thuận mạnh nhất với giá nhà (MedHouseVal) với hệ số 0.69 . Điều này cho thấy MedInc là biến dự báo (predictor) quan trọng nhất cho mô hình Hồi quy.

Vấn đề đa cộng tuyến: Ta thấy số phòng trung bình (AveRooms) và số phòng ngủ trung bình (AveBedrms) có tương quan rất cao (0.85). Trong thuật toán Hồi quy tuyến tính, việc giữ cả hai biến này có thể gây nhiễu mô hình. Ta giải quyết vấn đề này bằng cách dùng kỹ thuật Feature Engineering để tạo ra biến mới (ví dụ: Tỷ lệ phòng ngủ/tổng số phòng) hoặc loại bỏ một trong hai.

Mối quan hệ địa lý: Latitude và Longitude có tương quan âm mạnh (-0.92). Điều này đơn giản phản ánh hình dáng địa lý kéo dài theo hướng Tây Bắc - Đông Nam của bang California.

Ngoài ra: Population và AveOccup hầu như không có tương quan tuyến tính với giá nhà (~ 0). Tuy nhiên, trong ML, các mối quan hệ phi tuyến tính vẫn có thể tồn tại mà ma trận này không thể hiện hết.

2. Phân phối vị trí địa lý (Geographical Distribution)



Hình 4.6 Phân phối vị trí địa lý ở California

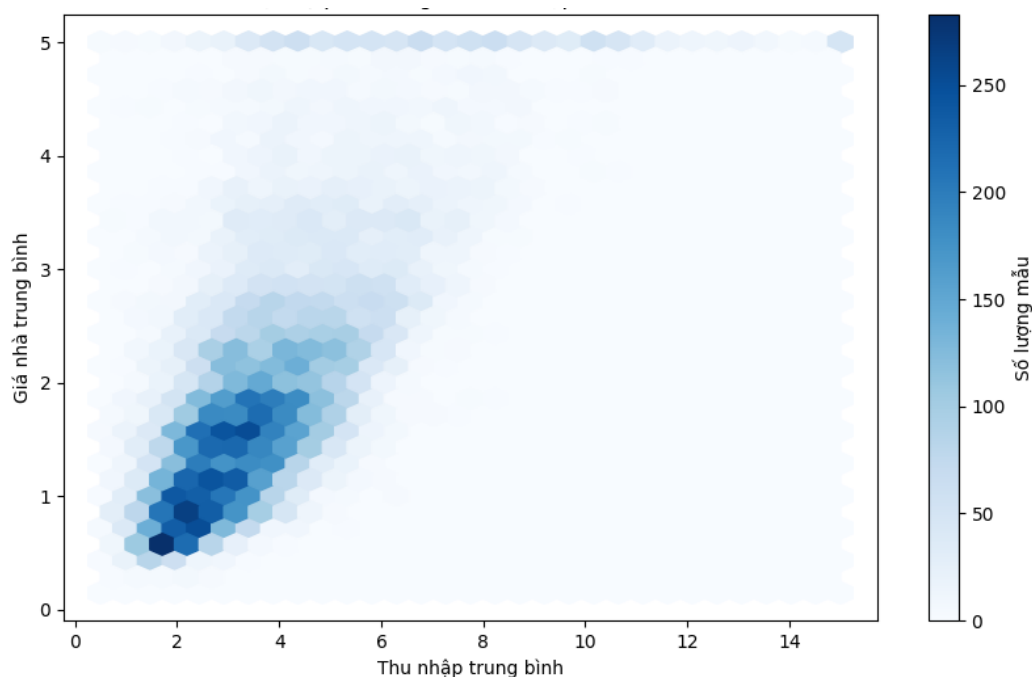
Yếu tố vị trí: Các điểm màu đỏ (giá nhà cao) tập trung dày đặc dọc theo đường bờ biển, đặc biệt là khu vực Vịnh San Francisco và Los Angeles.

Mật độ dân số: Kích thước các điểm (Population) lớn hơn cũng tập trung ở những khu vực đất đỏ này.

Tính phi tuyến tính: Giá nhà không tăng dần theo kinh độ hay vĩ độ một cách đơn giản, mà nó phụ thuộc vào khoảng cách tới các trung tâm kinh tế/ biển. Giá nhà không tăng dần đều theo Kinh độ hay Vĩ độ một cách đơn giản (ví dụ: Không phải cứ đi về phía Bắc là nhà đắt lên).

Từ vấn đề mối quan hệ phi tuyến tính nêu trên, nên nếu ta chỉ dùng thuật toán Hồi quy tuyến tính đơn thuần với hai biến Lat/ Long mà không có các biến hỗ trợ khác, mô hình sẽ khó đạt độ chính xác cao.

3. Biểu đồ phân tán (Scatter Plot) giữa Thu nhập và Giá nhà



Hình 4.7 Biểu đồ phân tán giữa Thu nhập và Giá nhà ở California

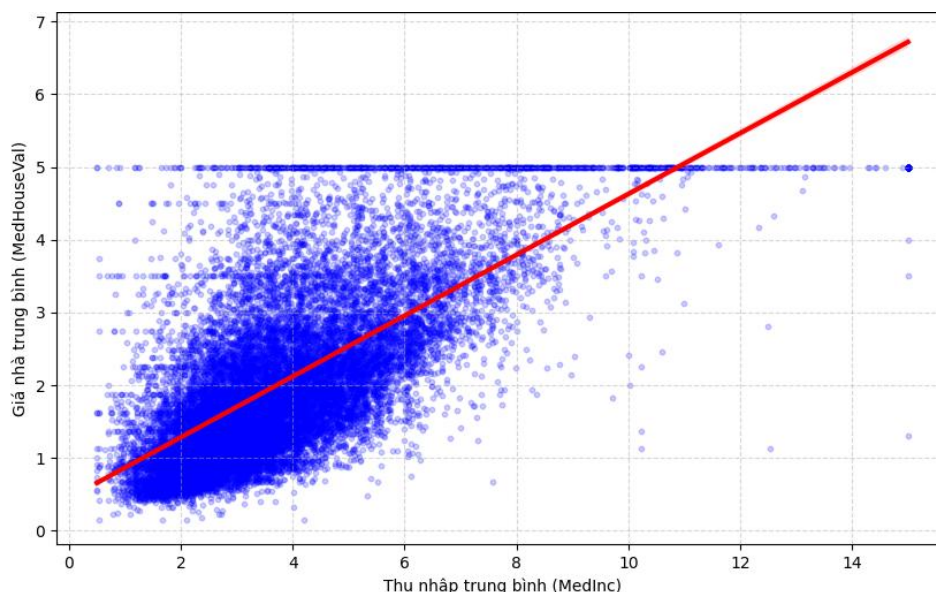
Biểu đồ này giúp đánh giá tính chất của mối quan hệ giữa biến độc lập quan trọng nhất và biến mục tiêu.

Xu hướng tuyến tính: Có một xu hướng đi lên rõ rệt: thu nhập càng cao, giá nhà càng cao. Phù hợp để sử dụng các mô hình Hồi quy.

Hiện tượng "Trần dữ liệu" (Data Capping): Dựa vào biểu đồ ta thấy một đường ngang đậm ở mức giá 5.0 (tương đương \$500,000). Nó thể hiện một vấn đề làm sạch dữ liệu. Giá trị đã bị giới hạn (capped) ở mức 5.0 trong quá trình thu thập. Khi huấn luyện mô hình ML, nếu không xử lý (loại bỏ hoặc gán nhãn lại), mô hình có thể bị "lệch" và dự báo kém chính xác ở phân khúc nhà ở cao cấp.

Mật độ dữ liệu: Các ô lục giác màu xanh đậm tập trung ở khoảng thu nhập từ 2 đến 4 và giá nhà từ 1 đến 2. Đây là phân khúc phổ biến nhất, nơi mô hình ML sẽ có nhiều dữ liệu để học nhất.

4. Biểu đồ đường hồi quy (Regression Line Plot)



Hình 4.8 Biểu đồ hồi quy giữa Thu nhập và Giá nhà ở California

Đây là biểu đồ Biểu đồ hồi quy (Regression Plot), một bước nâng cấp từ biểu đồ phân tán (scatter plot) đã có trước đó:

Mô hình hóa mối quan hệ tuyến tính: Đường thẳng màu đỏ chính là kết quả của thuật toán Hồi quy tuyến tính. Nó đại diện cho hàm số tối ưu nhất dự báo giá nhà dựa trên thu nhập. Độ dốc dương khẳng định một quy luật kinh tế cơ bản: Thu nhập là biến dự báo tỷ lệ thuận mạnh mẽ nhất với giá trị bất động sản. Bên cạnh đó đường đỏ này cố gắng cực tiểu hóa tổng bình phương sai số (Residuals) giữa dự đoán và thực tế. Khoảng mờ bao quanh đường thẳng là khoảng tin cậy (Confidence Interval), cho thấy mức độ chắc chắn của mô hình về đường hồi quy này.

Hiện tượng "Trần dữ liệu" (Data Capping) và nhiễu: Các điểm dữ liệu bị "chặn đầu" tại 5.0 tạo nên một ranh giới và đường màu đỏ bị kéo lên, đâm xuyên qua nó. Đây là một vấn đề lớn, nếu giữ nguyên dữ liệu này, mô hình sẽ học sai quy luật ở những vùng thu nhập cực cao.

Phương sai thay đổi (Heteroscedasticity): Ở mức thu nhập thấp ($MedInc < 4$), các điểm dữ liệu tập trung khá sát đường hồi quy. Khi thu nhập tăng lên, các điểm bắt đầu "xòe" rộng ra (phương sai của sai số tăng dần). Điều này cho thấy sai số của mô hình không đồng nhất trên toàn bộ tập dữ liệu. Để giải vấn đề này ta sử dụng thêm những thuật toán mạnh mẽ hơn như Random Forest để xử lý sự biến thiên phức tạp này.

4.2.3. Trực quan hóa dữ liệu bộ Credit Card Fraud Detection

1. Biểu đồ Tương quan (Correlation Heatmap)



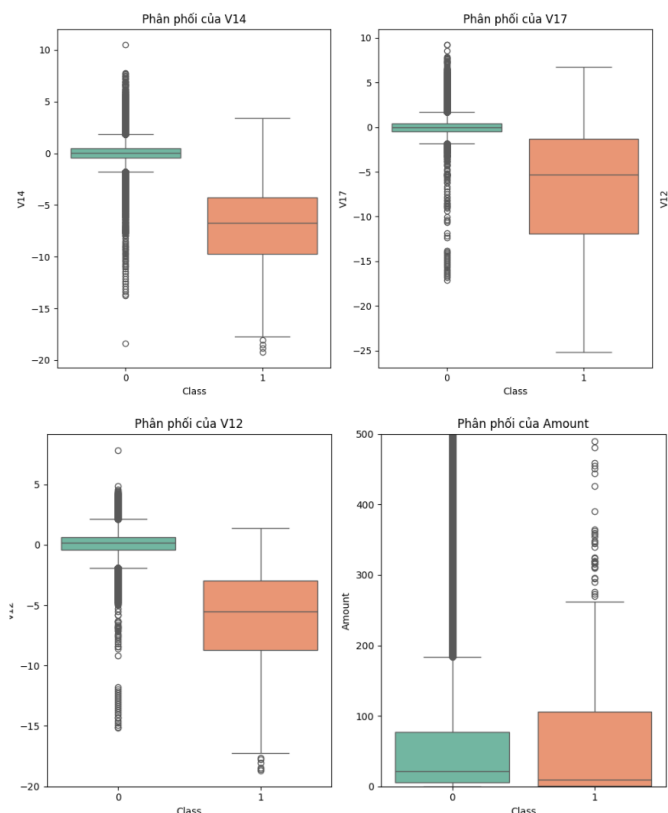
Hình 4.9 Biểu đồ Tương quan của Card Fraud Detection

Xét về độ tương quan cao: Mặc dù trong mô hình ML không quan trọng là đại lượng tương quan âm hay dương, mà nó chỉ quan tâm giá trị tương quan của của chúng, vì giá trị tương quan càng cao đồng nghĩa với mức độ rủi ro (gian lận) càng lớn. Dù vậy ta vẫn sẽ lưu ý chúng (ví dụ: Tương quan dương cao như *V11*, *V4*, *V2* và tương quan âm cao như *V17*, *V14*, *V12*) để lựa chọn những giá trị phù hợp để đưa vào chương trình. Độ tương quan âm càng mạnh – giá trị thấp bất thường của chúng là dấu hiệu cảnh báo cao.

Các đặc trưng ít tương quan: Các đặc trưng ở giữa biểu đồ có hệ số tương quan rất thấp (~ 0), có nghĩa là chúng ít có khả năng phân biệt giữa giao dịch gian lận và bình thường khi xét một mình.

Trường hợp của "Amount" (Số tiền): Có tương quan cực thấp (*0.006*). Điều này có vẻ mâu thuẫn nhưng lại rất phổ biến trong gian lận thẻ tín dụng, chính là gian lận không nhất thiết phải là những giao dịch số tiền lớn. Cũng giúp bác bỏ quan niệm sai lầm phổ biến cho rằng số tiền lớn luôn là gian lận.

2. Biểu đồ Boxplot so sánh đặc trưng (Phân phối theo Lớp)



Hình 4.10 Biểu đồ Boxplot so sánh 4 đặc trưng: V14, V17, V12, Amount

Chú thích:

Giữa hai lớp giao dịch: 0 (Bình thường - màu xanh) và 1 (Gian lận - màu cam).

"Hộp" thể hiện 50% dữ liệu ở giữa, vạch ngang trong hộp là Median (trung vị).

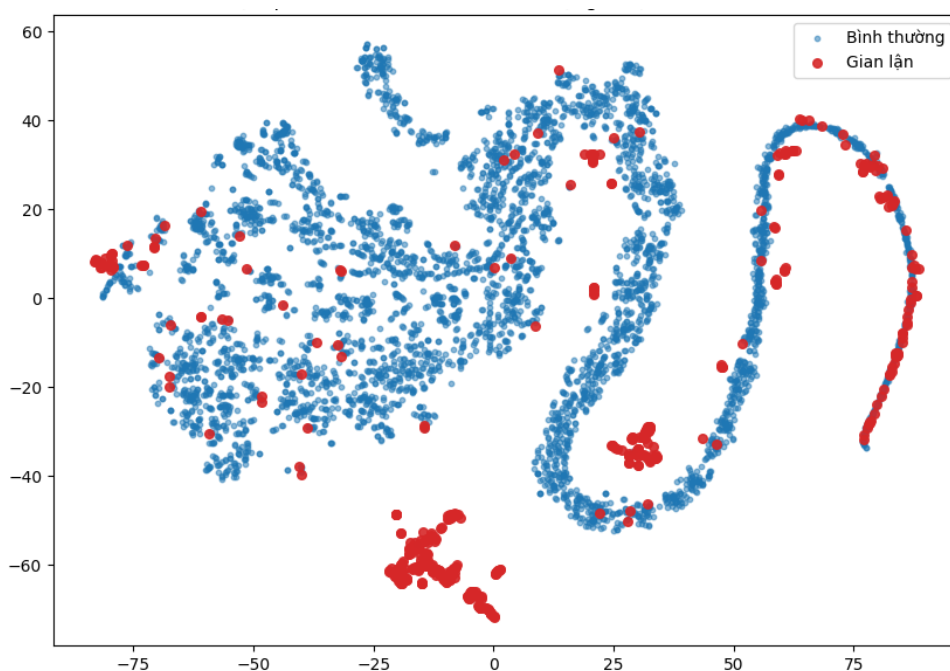
Các điểm nằm ngoài "râu" là điểm ngoại lai (outliers).

Về biểu đồ:

V14, V17, V12 (Các đặc trưng tương quan âm mạnh): Tổng quan thì hộp cam nằm thấp hơn hẳn so với hộp xanh, Median của nhóm gian lận thấp hơn nhiều so với nhóm bình thường. Điều này cho thấy giá trị thấp (âm sâu) là dấu hiệu cực mạnh của gian lận. Số lượng điểm outliers bên dưới nhóm gian lận lớn, dữ liệu rất mất cân bằng và phức tạp.

Amount (Số tiền): Phân phối của nhóm gian lận và bình thường trùng lặp nhau rất nhiều. Hộp của nhóm gian lận chỉ cao hơn một chút không đáng kể. Có rất nhiều điểm ngoại lai cực cao ở nhóm bình thường. Điều này có nghĩa là phần lớn các giao dịch có số tiền rất lớn lại là giao dịch bình thường. Điều này chỉ ra rõ số tiền giao dịch không phải là một đặc trưng mạnh để phân biệt gian lận khi xét một mình.

3. Biểu đồ Phân tán 2D sử dụng t-SNE



Hình 4.11 Biểu đồ Phân tán - t-SNE

Biểu đồ t-SNE (t-Distributed Stochastic Neighbor Embedding) là một kỹ thuật giảm chiều dữ liệu mạnh mẽ. Nó chuyển đổi dữ liệu nhiều chiều thành không gian 2D để trực quan hóa. Màu sắc thể hiện lớp: Màu xanh (Bình thường), Màu đỏ (Gian lận).

Sự tách biệt của các lớp: Các điểm màu đỏ tập trung thành các cụm (clusters) riêng biệt và tách rời khỏi các cụm điểm màu xanh. Nó chứng minh rằng ngay cả với dữ liệu mất cân bằng nghiêm trọng, các đặc trưng trong tập dữ liệu vẫn chứa đủ thông tin để phân biệt gian lận.

Hiệu quả phân cụm: Sự hình thành các cụm đỏ riêng lẻ cho thấy mô hình t-SNE có khả năng nhận diện các "khu vực" có xác suất gian lận cao.

Nhiều cụm gian lận khác nhau: Có nhiều cụm màu đỏ nhỏ nằm ở các vị trí khác nhau trên biểu đồ. Điều này cho thấy có nhiều loại hành vi gian lận khác nhau chứ không chỉ có một loại duy nhất. Mỗi cụm đỏ có thể đại diện cho một mẫu (pattern) gian lận cụ thể (ví dụ: một nhóm thẻ bị đánh cắp và sử dụng cùng một cách). Sự xuất hiện của nhiều cụm đỏ chứng minh tính hiệu quả của t-SNE trong việc "bắt" các hành vi gian lận khác nhau, một thách thức lớn trong thực tế.

Sự trộn lẫn: Vẫn có một số điểm đỏ nằm rải rác bên trong các cụm màu xanh, nhưng số lượng rất ít. Điều này cho thấy mô hình không hoàn hảo, nhưng rất hiệu quả.

4.2.4. Làm sạch dữ liệu:

Sau khi khám phá dữ liệu, bước tiếp theo là xử lý các vấn đề đã phát hiện để đảm bảo mô hình huấn luyện đạt hiệu quả cao nhất. Dựa trên kết quả khám phá nêu trên, cả hai bộ dữ liệu *California Housing* và *Credit Card Fraud Detection* đều không ghi nhận giá trị thiếu (Null/Missing values). Do đó, nghiên cứu này không cần thực hiện các kỹ thuật gán giá trị (Imputation).

1. Chia tỷ lệ đặc trưng cho bộ dữ liệu California Housing

Qua thống kê mô tả, dữ liệu tồn tại sự chênh lệch thang đo cực lớn. Biến *MedInc* có giá trị tối đa chỉ là 15.0, trong khi *Population* lên đến 35,682.

Để giải quyết vấn đề trên, ta sử dụng kỹ thuật Standardization (Z-score Scaling). Kỹ thuật này biến đổi dữ liệu sao cho mỗi đặc trưng đều có giá trị trung bình (μ) bằng 0 và độ lệch chuẩn (σ) bằng 1. Ta có công thức:

$$z = \frac{x - \mu}{\sigma}$$

Trong đó:

z : Giá trị sau khi đã được chuẩn hóa.

x : Giá trị gốc của đặc trưng.

μ : Giá trị trung bình của đặc trưng đó.

σ : Độ lệch chuẩn của đặc trưng đó.

Kết quả trước và sau xử lý:

SO SÁNH TRƯỚC VÀ SAU KHI SCALING				
	mean	std	min	max
Gốc_MedInc	3.870671e+00	1.899822	0.499900	15.000100
Gốc_Population	1.425477e+03	1132.462122	3.000000	35682.000000
Sau_Scaling_MedInc	5.508083e-17	1.000024	-1.774299	5.858286
Sau_Scaling_Population	-1.101617e-17	1.000024	-1.256123	30.250330

Hình 4.12 Kết quả trước và sau xử lý các biến

Mean (Trung bình): Sẽ là một số cực nhỏ (ví dụ: $-1.23e-16$), con số này thực chất là 0.

Std (Độ lệch chuẩn): Sẽ bằng 1.0.

Min/Max: Bây giờ Population không còn là 35,000 nữa mà sẽ chỉ dao động quanh khoảng -2 đến 3 (hoặc cao hơn một chút nếu có outlier)

2. Chia tỷ lệ đặc trưng cho bộ dữ liệu Credit Card Fraud Detection

Khái quát vấn đề, bộ dữ liệu này có cấu trúc hỗn hợp: Các biến $V1 - V28$ đã được xử lý qua PCA (có thang đo nhỏ), nhưng biến $Amount$ (Số tiền) lại là dữ liệu thô với giá trị rất lớn (lên đến 25,691). Thuật toán DBSCAN dựa trên khoảng cách *Euclidean* sẽ bị chi phối hoàn toàn bởi trục $Amount$, làm lu mờ các đặc trưng ẩn quan trọng khác.

Để giải quyết vấn đề: Ta áp dụng Chuẩn hóa chọn lọc (Selective Scaling) bằng *StandardScaler* chỉ cho riêng cột $Amount$ (và $Time$ nếu có) để đồng bộ hóa với các biến V đã qua PCA. Công thức khoảng cách Euclidean:

$$d(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

Trong đó:

$d(A, B)$: Khoảng cách (Distance) giữa giao dịch A và giao dịch B .

$(a_i - b_i)^2$: Sự khác biệt giữa đặc trưng thứ i của hai giao dịch, sau đó bình phương lên (để khoảng cách luôn là số dương, không bị âm).

Kết quả:

--- BẢNG DỮ LIỆU SAU KHI CHUẨN HÓA ---										
scaled_amount	V1	V2	V3	V4	V5	...	V27	V28	Class	
0.244964	-1.359807	-0.072781	2.536347	1.378155	-0.338321	...	0.133558	-0.021053	0	
-0.342475	1.191857	0.266151	0.166480	0.448154	0.060018	...	-0.008983	0.014724	0	
1.160686	-1.358354	-1.340163	1.773209	0.379780	-0.503198	...	-0.055353	-0.059752	0	
0.140534	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	...	0.062723	0.061458	0	
-0.073403	-1.158233	0.877737	1.548718	0.403034	-0.407193	...	0.219422	0.215153	0	

Hình 4.13 Bảng dữ liệu sau khi chuẩn hóa $Amount$

Dữ liệu sau chỉnh sửa thể hiện tính đồng nhất về mặt hình học. Biến $scaled_amount$ hiện tại có cùng biên độ dao động với các biến V , cho phép các thuật toán phân cụm (DBSCAN) và phân loại (Random Forest) khai thác tối đa thông tin từ tất cả các chiều dữ liệu mà không bị thiên kiến bởi các con số giá trị lớn.

4.3. Huấn luyện và Kết quả

4.3.1. Thiết lập thực nghiệm

Trước khi tiến hành huấn luyện các mô hình học máy, bước thiết lập thực nghiệm đóng vai trò nền tảng để đánh giá khách quan hiệu năng của thuật toán. Quy trình này tập trung vào việc chuẩn bị môi trường thử nghiệm sao cho mô hình không chỉ "học thuộc lòng" dữ liệu cũ mà còn có khả năng dự báo chính xác trên các dữ liệu mới chưa từng tiếp xúc. Ta sẽ chia tập dữ liệu theo 80/20, trong đó, 80% là tập huấn luyện để các mô hình như Hồi quy tuyến tính và Random Forest học tập các mẫu ẩn và điều chỉnh tham số nội bộ. Còn 20% là tập kiểm thử khác hoàn toàn so với tập trước đó, được dùng để mô phỏng dữ liệu thực tế.

Trong thực nghiệm này, tham số `random_sate` (thường được cố định là 42) trong mô hình Random Forest đóng vai trò quan trọng, việc cố định một "hạt giống" (seed) giúp kết quả thực nghiệm luôn đồng nhất sau mỗi lần chạy code. Đảm bảo rằng mọi sự thay đổi về hiệu suất của mô hình (nếu có) đều đến từ việc tinh chỉnh thuật toán chứ không phải do sự thay đổi ngẫu nhiên của tập dữ liệu huấn luyện.

Kết quả chia tập dữ liệu California Housing:

```
-----  
KẾT QUẢ CHIA TẬP DỮ LIỆU CALIFORNIA HOUSING  
-----  
Tổng số mẫu dữ liệu: 20640  
Số mẫu tập Huấn luyện (Train): 16512 (80%)  
Số mẫu tập Kiểm thử (Test):    4128 (20%)  
-----
```

Hình 4.14 Kết quả chia tập dữ liệu California Housing

Tập dữ liệu California Housing với tổng số 20.640 bản ghi đã được phân tách thành công theo tỷ lệ 80/20. Trong đó, 16.512 mẫu được dùng để huấn luyện mô hình nhằm chất lọc tri thức từ các đặc trưng địa lý và thu nhập. 4.128 mẫu còn lại đóng vai trò là dữ liệu mới hoàn toàn để đánh giá khả năng dự báo giá nhà của thuật toán trước khi đưa vào ứng dụng thực tiễn. Số lượng mẫu kiểm này đủ lớn để đưa ra một thước đo khách quan về sai số MSE và hệ số R^2 mà không lo bị ảnh hưởng bởi tính ngẫu nhiên của một vài điểm dữ liệu.

Kết quả chia tập dữ liệu Credit Card Fraud Detection:

```

-----
KẾT QUẢ CHIA TẬP DỮ LIỆU (STRATIFIED SPLIT)
-----
Số lượng mẫu tập Huấn luyện (Train): 227845
Số lượng mẫu tập Kiểm thử (Test): 56962

Tỷ lệ gian lận trong tập Train: 0.1729%
Tỷ lệ gian lận trong tập Test: 0.1720%
-----

```

Hình 4.15 Kết quả chia tập dữ liệu Credit Card Fraud Detection

Với 227.845 mẫu ở tập Huấn luyện và 56.962 mẫu ở tập Kiểm thử, máy tính đã thực hiện chính xác việc chia dữ liệu theo tỷ lệ 80/20 trên tổng số 284.807 giao dịch. Với số lượng mẫu ở tập huấn luyện đây là một con số hợp lý để thuật toán Random Forest xây dựng hàng trăm cây quyết định mà không sợ bị thiếu thông tin.

Dựa trên tỷ lệ gian lận trong cả hai tập, có thể thấy đây là thành công trong việc triển khai kỹ thuật Stratified Split. Vì dữ liệu được chia tách nhưng vẫn bảo toàn nguyên vẹn đặc tính mất cân bằng của bài toán với tỷ lệ gian lận xấp xỉ 0.172% ở cả hai tập Huấn luyện và Kiểm thử. Điều này đảm bảo rằng mô hình không chỉ được đào tạo trên một tập mẫu đại diện mà còn được thẩm định một cách công bằng, tránh hiện tượng sai lệch đánh giá do sự phân bố ngẫu nhiên của các ca gian lận cực hiếm.

4.3.2. Quá trình huấn luyện và Kết quả Hồi quy (California Housing)

Trong phần này, thuật toán Hồi quy tuyến tính được triển khai để xây dựng mô hình dự báo giá trị trung bình của các ngôi nhà dựa trên 8 đặc trưng đầu vào đã được chuẩn hóa.

1. Quá trình thực hiện

Mô hình được huấn luyện bằng thư viện scikit-learn thông qua lớp LinearRegression. Quá trình bao gồm:

Khởi tạo: Thiết lập mô hình hồi quy dựa trên phương pháp OLS.

Huấn luyện: Sử dụng hàm `.fit()` trên tập dữ liệu X_{train_h} và y_{train_h} .

Dự báo: Sau khi học xong các trọng số, mô hình thực hiện dự báo trên tập X_{test_h} để kiểm tra sai số.

2. Kết quả

KẾT QUẢ MÔ HÌNH CALIFORNIA HOUSING	
Mean Squared Error (MSE):	0.5559
R2 Score (Độ chính xác):	0.5758
Thời gian huấn luyện:	0.008328 giây

Hình 4.16 Kết quả Hồi quy - California Housing

Về sai số (MSE): Giá trị 0.5559 cho thấy mức độ chênh lệch giữa giá nhà dự báo và giá thực tế nằm trong khoảng chấp nhận được đối với một mô hình cơ sở (Baseline model). Do đơn vị của biến mục tiêu là 100.000 USD, sai số này phản ánh các biến động thực tế của thị trường bất động sản California.

Về khả năng giải thích (R^2): Chỉ số 0.5758 (~58%) khẳng định rằng hơn một nửa sự biến động của giá nhà có thể được giải thích thông qua các yếu tố kinh tế - địa lý như thu nhập và vị trí. Tuy nhiên, 42% còn lại chính là "khoảng tối" của các biến số mà mô hình tuyến tính chưa chạm tới, như tiện ích công cộng, hạ tầng giao thông hoặc tâm lý thị trường.

Tác động của hiện tượng “Trần dữ liệu” (Data Capping): Việc dữ liệu bị giới hạn ở mức 5.0 (tương đương 500,000 USD) làm cho đường hồi quy bị "kéo lệch". Trên thực tế, ở phân khúc cao cấp, mối quan hệ giữa thu nhập và giá nhà có thể không còn là đường thẳng nữa mà tăng theo hàm mũ. Đây là lý do tại sao sai số MSE vẫn còn ở mức 0.55 – mô hình đang cố gắng "khớp" một đường thẳng vào một tập dữ liệu bị chặn ở đỉnh.

Về hiệu suất: Thời gian huấn luyện chưa tới 0.01 giây chứng minh hiệu quả cực cao của phương pháp Bình phương tối thiểu (OLS) khi được tối ưu hóa qua các phép toán ma trận trên phần cứng.

4.3.3. Quá trình huấn luyện và Kết quả Phân lớp (Fraud Detection)

Thực nghiệm trên bộ dữ liệu Credit Card Fraud Detection bằng thuật toán Random Forest đã cho phản ánh đúng bản chất của một hệ thống phát hiện gian lận trong môi trường thực tế.

1. Quá trình thực hiện

Thuật toán Rừng ngẫu nhiên (Random Forest) được triển khai thông qua thư viện scikit-learn để xây dựng mô hình phân lớp nhị phân.

Khởi tạo: Thiết lập mô hình Rừng ngẫu nhiên với 100 cây quyết định. Đặc biệt, tham số `class_weight='balanced'` được kích hoạt để xử lý sự mất cân bằng lớp (chỉ 0.17% gian lận), giúp mô hình tập trung học các đặc trưng của nhóm thiểu số hiệu quả hơn.

Huấn luyện: Sử dụng hàm `.fit()` trên tập dữ liệu huấn luyện đã phân tầng (Stratified) gồm 227.845 mẫu. Trong giai đoạn này, thuật toán thực hiện kỹ thuật lấy mẫu Bootstrap và chọn ngẫu nhiên các tập con đặc trưng để xây dựng các cây quyết định độc lập, nhằm tối ưu hóa khả năng chống nhiễu.

Dự báo: Sau khi hệ thống tập hợp (Ensemble) hoàn tất quá trình học, mô hình thực hiện dự báo nhãn lớp (0 hoặc 1) trên tập kiểm thử độc lập gồm 56.962 mẫu. Kết quả cuối cùng được tổng hợp từ sự biểu quyết đa số của toàn bộ các cây trong rừng để trích xuất ra ma trận nhầm lẫn và các chỉ số đánh giá.

2. Kết quả

```
-----
Thời gian huấn luyện: 10.17 giây
Độ chính xác tổng thể (Accuracy): 0.9995

BÁO CÁO CHI TIẾT (Classification Report):

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.96	0.77	0.85	98
accuracy			1.00	56962
macro avg	0.98	0.88	0.93	56962
weighted avg	1.00	1.00	1.00	56962

```
-----
```

Hình 4.17 Kết quả của mô hình Random Forest

Accuracy (Độ chính xác): Đạt 0.9995 cho thấy độ chính xác tổng thể trên toàn bộ tập kiểm thử. Kết quả này gần như vô nghĩa vì dữ liệu quá mất cân bằng (chỉ 0.17% là gian lận). Vậy nên ta cần ma trận nhầm lẫn (Hình 4.18) để làm rõ hơn về mẫu dữ liệu này.

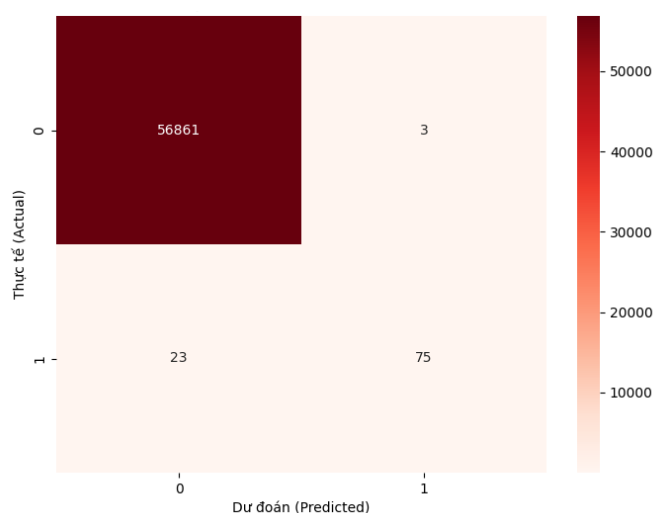
Precision (Độ tin cậy) – Lớp 1: Trong các ca máy báo gian lận, có 96% là đúng.

Recall (Độ tin cậy) - Lớp 1: Máy đã "quét" được 77% tổng số vụ gian lận thực tế. Mặc dù Precision rất cao (giảm thiểu sai sót cho khách hàng), nhưng việc bỏ sót 23 ca gian lận cho thấy cần nghiên cứu thêm các kỹ thuật như lấy mẫu lại (Resampling) hoặc tinh chỉnh sâu hơn các siêu tham số của rừng cây để nâng cao chỉ số Recall trong tương lai.

F1-Score (Chỉ số cân bằng) – Lớp 1: Giữa Precision và Recall cho lớp gian lận là 0.85. Mặc dù tỷ lệ gian lận cực thấp (~ 0.17%) nên việc đạt được Chỉ số cân bằng cao như

vậy, cho thấy thuật toán Random Forest cùng kỹ thuật *class_weight='balanced'* đã hoạt động cực kỳ hiệu quả, không bị "đánh lừa" bởi số lượng áp đảo của các giao dịch bình thường.

Thời gian huấn luyện: 10.17 cho thấy tính thực tiễn cao cho thuật toán cho hơn 200.000 dòng dữ liệu minh chứng cho khả năng ứng dụng của Random Forest trong các hệ thống giám sát giao dịch theo thời gian thực tại ngân hàng.



Hình 4.18 Ma trận nhầm lẫn của mô hình Random Forest

Từ ma trận trên và (Hình 4.17-Kết quả của mô hình Random Forest) chúng ta có cái nhìn chi tiết hơn về cách mô hình đưa ra quyết định trên 56.962 giao dịch kiểm thử:

Lá chắn bảo vệ khách hàng (Precision = 0.96): Trong tổng số giao dịch, chỉ có 3 trường hợp bị cảnh báo nhầm (False Positive) trên hơn 56,000 giao dịch. Điều này cực kỳ quan trọng đối với trải nghiệm người dùng: khách hàng sẽ không cảm thấy phiền toái vì thẻ bị khóa oan.

Rủi ro tài chính (Recall = 0.77): Mô hình đã bỏ lọt 23 vụ gian lận (False Negative). Dù con số 23 nghe có vẻ nhỏ, nhưng trong tài chính, mỗi vụ lọt lưới có thể gây thiệt hại hàng tỷ đồng. Đây là "gót chân Achilles" mà bạn cần nhấn mạnh khi đề xuất hướng phát triển: cần tăng cường Recall ngay cả khi phải hy sinh một chút Precision.

F1-Score 0.85: Đây mới là con số chứng minh thuật toán Rừng ngẫu nhiên (với kỹ thuật trọng số lớp cân bằng) đã vượt qua được thách thức của dữ liệu mất cân bằng để đưa ra quyết định thông minh nhất.

4.3.4. Quá trình huấn luyện và Kết quả Phân cụm (DBSCAN)

Thuật toán DBSCAN được triển khai như một phương pháp học không giám sát để phát hiện các mẫu hành vi lạ và các điểm dị biệt (outliers) trong dữ liệu giao dịch.

1. Quá trình thực hiện

Khởi tạo: Thiết lập mô hình DBSCAN với tham số bán kính ϵ (Epsilon) và số điểm tối thiểu (*min_samples*) phù hợp để định nghĩa vùng mật độ cao. Mục tiêu là cô lập các giao dịch không thuộc về bất kỳ nhóm hành vi chuẩn nào.

Huấn luyện: Áp dụng hàm *fit_predict()* lên bộ dữ liệu đã được chuẩn hóa (thường thực hiện trên một tập mẫu đại diện để đảm bảo hiệu suất tính toán). Thuật toán thực hiện lan truyền mật độ để xác định các điểm lõi và điểm biên.

Phân loại: Các điểm dữ liệu nằm trong vùng mật độ thấp, không thể kết nối mật độ với các cụm chính, sẽ được thuật toán gán nhãn -1 (điểm nhiễu). Đây chính là các đối tượng tiềm ẩn hành vi gian lận.

2. Kết quả

```
-----  
Thời gian huấn luyện: 2.4880 giây  
Số lượng cụm tìm thấy: 31  
Số lượng điểm nhiễu (Outliers): 7712  
-----
```

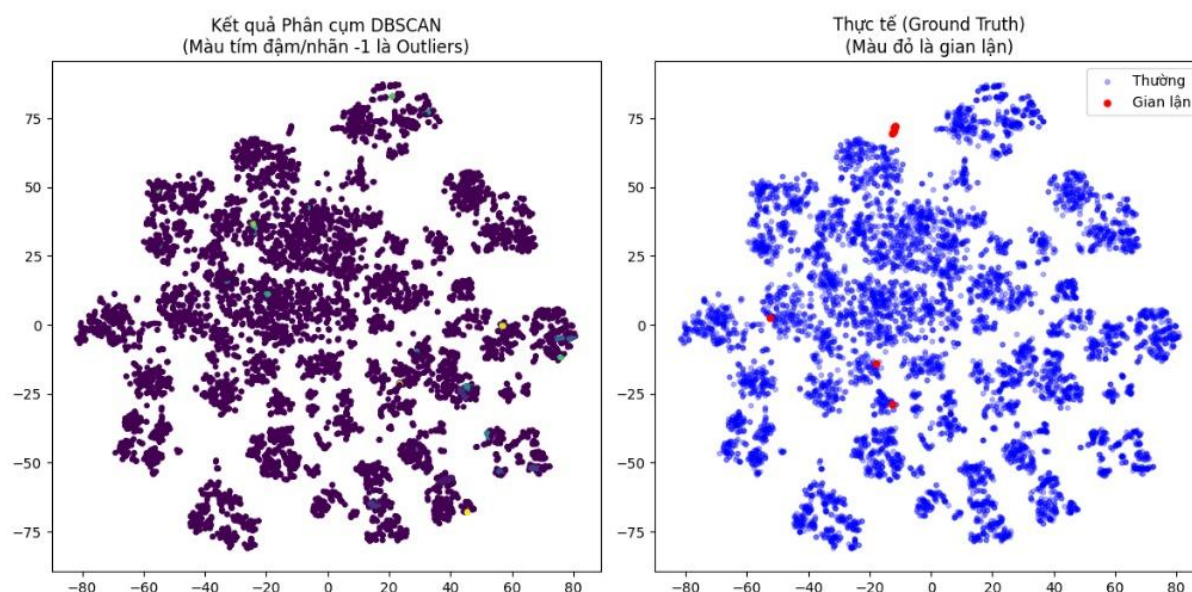
Hình 4.19 Mô hình DBSCAN

Thời gian huấn luyện: 2.4880 giây. Cho thấy tốc độ xử lý này rất ấn tượng đối với một thuật toán dựa trên mật độ và khoảng cách hình học. Điều này cho thấy DBSCAN có khả năng phản hồi nhanh chóng, hỗ trợ đắc lực cho việc phân tích các khối dữ liệu giao dịch khổng lồ trong thời gian ngắn.

Nhận diện 31 cụm hành vi: Con số này phản ánh thế giới thực không chỉ có "bình thường" và "gian lận". Nó có 31 kiểu chi tiêu khác nhau (như người mua sắm đêm, người du lịch quốc tế, người chi tiêu nhỏ lẻ...).

7,712 điểm nhiễu (Outliers): Thuật toán đã cô lập một lượng lớn dữ liệu nằm ngoài các quy chuẩn thông thường. Trong số này, có những điểm trùng khớp với gian lận thực tế, nhưng quan trọng hơn là nó phát hiện được những hành vi lạ mới phát sinh (chưa có

trong lịch sử dân nhân). Đây chính là giá trị của học không giám sát: tìm ra "kẻ gian" ngay cả khi chưa biết mặt mũi chúng ra sao.



Hình 4.20 Kết quả phân cụm DBSCAN và thực tế (Ground Truth)

Sự tương đồng về không gian: Nhìn vào biểu đồ t-SNE, các điểm màu đỏ ở hình “*thực tế*” hầu hết đều rơi vào các khu vực mà DBSCAN đánh dấu là "Outliers" (màu tím đậm/nhân -1) ở hình “*phân cụm DBSCAN*”. Điều này minh chứng cho tính hiệu quả của DBSCAN trong việc nhận diện gian lận dựa trên mật độ hình học.

Khả năng phát hiện mẫu mới: Việc tìm thấy 31 cụm khác nhau cho thấy dữ liệu giao dịch rất phức tạp. DBSCAN đã thành công trong việc gom nhóm các hành vi bình thường và tách biệt các ca gian lận rải rác.

Ưu điểm vượt trội: Kết quả này khẳng định DBSCAN có thể phát hiện gian lận mà không cần dữ liệu được dán nhãn trước, giúp hệ thống ngân hàng tìm ra được những kiểu lừa đảo mới chưa từng có trong lịch sử.

4.4. Nhận xét

Sau khi triển khai thực nghiệm trên hai bộ dữ liệu khác nhau, phần này thực hiện so sánh đối chiếu giữa ba thuật toán nghiên cứu (Hồi quy tuyến tính, Random Forest và DBSCAN) dựa trên các tiêu chí kỹ thuật và hiệu quả thực tế.

4.4.1. So sánh hiệu năng

Bảng 4.3 So sánh hiệu năng giữa các thuật toán thực nghiệm

Tiêu chí	Hồi quy tuyến tính	Random Forest	DBSCAN
Loại hình học	Có giám sát	Có giám sát	Không giám sát
Bài toán áp dụng	Dự báo giá nhà (Liên tục)	Phát hiện gian lận (Phân lớp)	Phát hiện bất thường (Phân cụm)
Thời gian huấn luyện	0.0083 giây (Nhanh nhất)	10.17 giây (Chậm nhất)	2.4880 giây (Trung bình)
Độ phức tạp	Thấp, dễ giải thích	Cao, khó giải thích từng cây	Trung bình, nhạy cảm với tham số ϵ
Xử lý nhiễu/Outlier	Kém, dễ bị lệch mô hình	Tốt, nhờ kỹ thuật Bagging	Rất tốt, chuyên biệt để lọc nhiễu

4.4.2. Giải thích kết quả

1. Về tốc độ và tài nguyên tính toán:

Hồi quy tuyến tính thể hiện ưu thế tuyệt đối về tốc độ (dưới 0.01 giây) nhờ công thức toán học xác định và kỹ thuật Vector hóa.

Random Forest tốn nhiều thời gian nhất (hơn 10 giây) vì phải xây dựng đồng thời 100 cây quyết định độc lập trên tập dữ liệu không lồ.

DBSCAN có tốc độ phản hồi ấn tượng đối với một thuật toán không giám sát, cho phép xử lý dữ liệu mật độ cao một cách linh hoạt.

2. Về khả năng đáp ứng đặc thù dữ liệu:

Hồi quy tuyến tính mặc dù nhanh nhưng lại bộc lộ hạn chế khi đối mặt với hiện tượng "Trần dữ liệu" và tính phi tuyến tính của địa lý trong bộ dữ liệu nhà ở, dẫn đến chỉ số R^2 chỉ dừng lại ở mức $\sim 58\%$.

Đối với dữ liệu có tính chất phức tạp và khó nắm bắt như gian lận thẻ tín dụng, Random Forest là công cụ mạnh mẽ nhất nhờ khả năng xử lý mất cân bằng lớp thông qua *class_weight='balanced'*, đạt chỉ số *F1-Score 0.85* ấn tượng.

DBSCAN chứng minh vai trò độc đáo khi tự động phát hiện được 7.712 điểm bất thường mà không cần biết trước nhãn, một khả năng mà các thuật toán có giám sát không có.

4.4.3. Kết luận

Mỗi thuật toán đều hoàn thành xuất sắc nhiệm vụ trong phạm vi bài toán được giao. Nếu Hồi quy tuyến tính là công cụ dự báo xu hướng nhanh chóng và tinh gọn, thì Random Forest là "lá chắn" tin cậy cho bài toán phân loại nhạy cảm. Trong khi đó, DBSCAN đóng vai trò giúp tìm ra những góc khuất bất thường trong dữ liệu mà các phương pháp truyền thống dễ dàng bỏ qua

4.5. Tóm tắt

Chương 4 tập trung thực nghiệm và đánh giá hiệu năng của ba thuật toán học máy trên hai tập dữ liệu thực tế là California Housing và Credit Card Fraud Detection . Nội dung bắt đầu từ bước khám phá dữ liệu, tiền xử lý chuẩn hóa thang đo đến việc chia tập dữ liệu huấn luyện và kiểm thử theo tỷ lệ 80/20 . Kết quả thực nghiệm minh chứng Hồi quy tuyến tính có ưu thế về tốc độ dự báo, Random Forest mạnh về phân loại gian lận, và DBSCAN hiệu quả trong việc phát hiện hành vi bất thường . Thông qua các chỉ số như MSE, R^2 , Precision và Recall, chương này cung cấp cái nhìn khách quan về độ chính xác và tính thực tiễn của từng mô hình. Cuối cùng, phần so sánh tổng hợp giúp xác định thuật toán tối ưu nhất cho từng đặc thù bài toán cụ thể .

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Tiểu luận đã hoàn thành việc nghiên cứu và triển khai thực nghiệm các thuật toán học máy tiêu biểu, đạt được các mục tiêu cốt lõi sau:

Hệ thống hóa lý thuyết: Làm rõ quy trình 7 bước xây dựng mô hình và cơ chế biểu diễn dữ liệu dưới dạng Tensor để tối ưu hóa hiệu suất trên phần cứng.

Hiệu quả dự báo (Hồi quy): Mô hình Hồi quy tuyến tính trên bộ dữ liệu California Housing cho tốc độ xử lý vượt trội (dưới 0.01 giây) với hệ số R^2 đạt ~58%, khẳng định thu nhập là yếu tố quyết định giá nhà.

Phát hiện gian lận (Phân lớp): Thuật toán Random Forest xử lý tốt dữ liệu mất cân bằng, đạt Precision 0.96 và F1-Score 0.85. Ma trận nhầm lẫn cho thấy khả năng bảo vệ khách hàng tốt khi chỉ có 3 ca cảnh báo sai trên tổng số hơn 56.000 giao dịch.

Khám phá hành vi (Phân cụm): Thuật toán DBSCAN chứng minh ưu thế của học không giám sát khi tự động nhận diện 31 cụm hành vi và cô lập 7.712 điểm nhiễu tiềm ẩn gian lận mà không cần dán nhãn dữ liệu.

2. Hướng phát triển

Để nâng cao tính ứng dụng và độ chính xác của hệ thống trong tương lai, nghiên cứu cần tập trung vào các hướng sau:

Tối ưu chỉ số Recall: Áp dụng kỹ thuật SMOTE hoặc các thuật toán Boosting để xử lý triệt để 23 ca gian lận bị bỏ sót, nhằm giảm thiểu tối đa rủi ro tài chính.

Nghiên cứu mô hình phi tuyến: Sử dụng Mạng nơ-ron (Deep Learning) để giải quyết các mối quan hệ địa lý phức tạp, giúp cải thiện chỉ số giải thích R^2 vốn còn 42% chưa được bao quát ở mô hình tuyến tính.

Triển khai thực tế: Tích hợp mô hình vào hệ thống giám sát thời gian thực để đưa ra cảnh báo gian lận ngay khi phát sinh giao dịch trên nền tảng ngân hàng số.

Mở rộng dữ liệu: Thử nghiệm trên các tập dữ liệu đa quốc gia và đa ngành nghề để kiểm chứng tính tổng quát hóa của các thuật toán.

TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] FPT IS. (2025). *Học máy (Machine Learning) là gì? Giải mã "bộ não nhân tạo" trong thế giới số*. Được truy lục từ FPT IS.
- [2] Thủy, N.T.(1995). Các phương pháp giải quyết vấn đề và kỹ thuật xử lý tri thức.

Tiếng Anh

- [3] *1.1. Linear Models*. (n.d.). Retrieved from scikit learn.
- [4] Adrien Payong, Abdeladim Fadheli . (2022). *Credit Card Fraud Detection in Python*. Retrieved from The Python Code.
- [5] Beers, B. (2026). *Regression: Definition, Analysis, Calculation, and Example*. Retrieved from Investopedia.
- [6] Bidgoli, H. (2003). Encyclopedia of Information Systems. In H. Bidgoli, *Encyclopedia of Information Systems* (pp. 33-41).
- [7] Bishop, C. (2006). Pattern Recognition and Machine Learning.
- [8] Donges, Niklas. (2024). *Random Forest: A Complete Guide for Machine Learning*. Retrieved from builtin.
- [9] Fangfang, L. (n.d.). *What is logistic regression?* Retrieved from IBM.
- [10] *Mean Squared Error*. (2026). Retrieved from geeksforgeeks.
- [11] Nugent, C. (2018). *California Housing Prices*. Retrieved from kaggle.
- [12] nvidia. (2023). *Matrix Multiplication Background*. Retrieved from nvidia.
- [13] Sharma, A. (2022). *Linear Regression in Machine Learning*. Retrieved from scaler.
- [14] *The California housing dataset*. (n.d.). Retrieved from inria.